



AI Agent 避坑指南

HUMAN DIRECTS, AI DELIVERS

从架构选择到治理失控，从记忆设计到组织认知
一本让 AI Agent 真正交付价值的实战手册



github.com/xg-gh-25/SwarmAI

XG · SwarmAI · 2026

本书内容基于个人实践与观点，不代表任何组织立场。所有案例来自单一系统的 N=1 验证，仅供参考。

目录

序章：你的 AI 越用越聪明，还是越用越一样？

第一章：设计哲学 — 自改进系统的六根支柱

- 1.1 预防优于恢复 — 三级硬化
 - 1.2 验证优于推断 — 置信度是反向信号
 - 1.3 知识即基础设施 — DDD 供养一切
 - 1.4 修正优于能力 — 失败是最高价值数据
 - 1.5 时间复利 — 系统从不是同一个 Agent
 - 1.6 所有权即架构 — 边界写在代码里
- 取舍 — 我们选择了什么、放弃了什么

Part 2：架构坑 — 选错了要推倒重来

- 2.1 Multi-Agent 是协作税，不是银弹
- 2.2 你不需要 Neo4j
- 2.3 "全自动"是幻觉
- 2.4 Session 结束 $\neq$ 空闲
- 2.5 一个 Agent 要有 6 个"自"

Part 3：记忆坑 — 没有记忆就没有理解

- 3.1 只靠 context window 不叫记忆
- 3.2 越用越聪明 $\neq$ 上下文越长
- 3.3 领域知识要自己长出来
- 3.4 $1+1+1+1 > 4$ 是编译器保证
- 3.5 Agent 是 teammate 不是工具

Part 4：治理坑 — AI 越聪明越需要约束

- 4.1 规则越多越不遵守
- 4.2 给 AI 个性，它就开始偷懒
- 4.3 "我很自信" = 跳过检查的借口
- 4.4 "以后再修" = AI 在拖延
- 4.5 给 AI 治理权限，它会用来规避治理

Part 5: 交付坑 — 能跑 ≠ 能用

5.1 "一句话进去 PR 出来"没那么简单

5.2 内容生产也可以是黑盒

5.3 AI for Data: 从幻觉到精准

Part 6: 认知坑 — 你的组织还没准备好

6.1 AI 转型失败是 S×T 错配

6.2 层级制是信息路由协议

6.3 创业四阶段没变, 瓶颈变了

6.4 "越用越聪明"是唯一采购标准

6.5 一个人 + AI = 一个团队

结语: Human Directs, AI Delivers

附录

A. 系统架构全景 — 参考实现 (9 子系统)

B. 自治五级完整定义

C. Discussion 索引

D. 三条阅读路径

序章

你的 AI 越用越聪明，还是越用越一样？

这本书写给谁

Builder: 你已经在用 AI 写代码 (Claude Code / Cursor / Copilot), 想让 Agent 从 "能用" 变成 "越用越好"。

Architect: 你在评估或设计 AI Agent 框架, 需要避开不可逆的架构错误。

决策者: 你在做 AI 工具采购或组织转型决策, 需要一个判断标准。

三条阅读路径见附录 D — Builder (~45min) / Architect (~60min) / CXO (~25min)。

打开你最常用的 AI 工具 — Claude Code、Cursor、ChatGPT、Copilot — 回答一个问题：

用了三个月后，它比第一天好了多少？

如果答案是"差不多", 你并不孤单。90% 的 AI 工具画出的价值曲线是平的：第一天惊艳，第三十天一样，第三百天还是一样。它不记得你的偏好，不了解你的代码库，不积累任何经验。每次打开，都是从零开始。

这就是 Flat 和 Compound 的区别。

两种价值曲线

	Flat (平的)	Compound (复利的)
第 1 天	惊艳	一般
第 30 天	一样惊艳	明显比第 1 天好
第 100 天	还是一样	好得让你不敢换
换模型	毫无影响	知识全部保留
护城河	无 (谁都能换)	累积的认知不可复制

Compound 系统的壁垒不是技术 — 是时间。每用一天，差距拉大一天。这不是功能，是架构选择。

Agent Harness：控制面 + 引擎

为什么多数 AI 工具是 Flat 的？因为它们只有引擎 (LLM)，没有控制面。

用自动驾驶来类比：

- 引擎 = LLM 本身 (Claude、GPT、Gemini) —— 它提供原始智能
- 控制面 = Harness —— 记忆、治理、进化、交付流程

一辆车只有发动机，没有方向盘、刹车、导航、行车记录仪 — 你敢让它自己开吗？

Agent Harness 就是这套控制系统。它决定了你的 AI 处在哪个自治级别：

L1 → L5：你现在在哪一级？

Level	名称	人类做什么	AI 做什么	典型产品
L1	Copilot	写代码	补全当前行	GitHub Copilot
L2	Assistant	给指令	执行单步	ChatGPT

Level	名称	人类做什么	AI 做什么	典型产品
L3	Operator	审批	执行多步任务	Claude Code, Cursor
L4	Autonomous	给方向	端到端交付 + 自审查	本书案例系统
L5	Self-Governing	定目标	自主决策 + 自我进化	尚不存在

关键洞察
从 L3 到 L4 的跃迁，不是模型更聪明了 — 是 Harness 更厚了。记忆、治理、自进化、知识沉淀 — 这些是 L4 必须有而 L3 没有的层。

本书接下来的内容就是：从 L3 到 L4 这段路上，我们踩过的 23 个坑。

本书地图

23 个坑分布在 5 个区域，全部指向同一套设计哲学（6 根支柱）：

Part	区域	核心问题	坑数
2	架构坑	选错了要推倒重来	5
3	记忆坑	没有记忆就没有理解	5
4	治理坑	AI 越聪明越需要约束	5
5	交付坑	能跑 ≠ 能用	3
6	认知坑	你的组织还没准备好	5

但在进入具体的坑之前，我们先讲支撑一切的设计哲学 — 六根支柱。

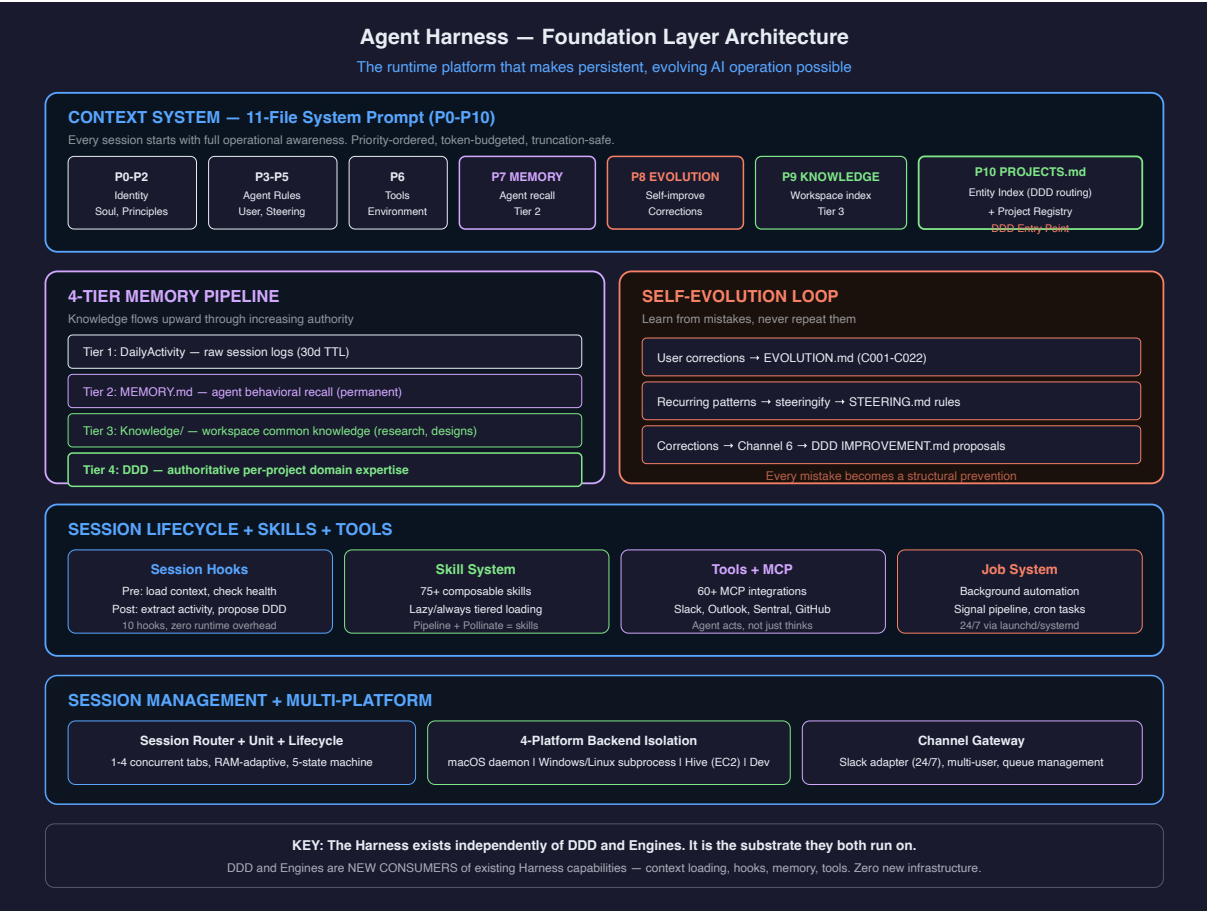


Figure 1: Agent Harness Architecture — Memory, Context, Pipeline, DDD, Evolution as infrastructure layers.

第一章

设计哲学 — 自改进系统的六根支柱

多数 AI 系统优化"现在能用"。我们优化"不可能以相同方式再次失败"。
这不是宣言。这是工程报告。每个论断可追溯到具体 commit、具体失败、或具体架构决策。

六根支柱之间不是独立的。去掉任何一根，其余都退化。这种相互依赖是刻意的架构赌注 — 也是让系统难以部分复制的原因。

1.1 预防优于恢复 — 三级硬化

核心信念：最好的修复让 bug 在结构上不可能发生，而不仅仅是不太可能。

多数团队对失败的反应是"下次注意"。那是 Level 1 思维。自觉不能规模化。门控可以。

我们系统的每条规则都始于文字指令。有些就留在那里。重要的升级为代码。关键的变成结构性不可能：

级别	机制	示例	绕过成本
L1 指令	写在规则文件里	"代码变更后必须跑测试"	零 — 靠自觉
L2 机械闸	代码阻断	对抗子代理必须运行才能 push	需要删除代码
L3 结构不可能	架构层面阻止	群组频道物理上无法访问私人记忆	需要重新设计系统

演进路径始终是渐进的：

发布 → 失败 → 修正(L1 文字) → 复发 → 门控(L2 代码) → 再复发 → 不可能(L3 架构)

真实案例

对抗审查始于一建议。到第 11 次修正时，pipeline 以 10/10 信心发布了 100% 不工作的代码 — 它变成书面规则。到第 21 次修正，变成强制代码门控。同一类别总共触发 8 次，直到结构强制终止复发。

为什么重要："我们会更小心"是最贵的谎言。小心是一次性资源，会被疲劳、自信、截止日期消耗殆尽。门控是永久的。

1.2 验证优于推断 — 置信度是反向信号

核心信念：你感觉越确定，越可能是基于陈旧推理而非新鲜证据。

这条原则来自痛苦的模式：

- 记忆连续 5 个会话声称"系统只有 L0 和 L1"。实际：L0–L4 全部实现（1,142 行，106 测试）
- 不读代码就断言系统行为。三次。
- 基于直觉"优化"了一个系统。测量表明"问题"根本不存在

三级断言标准：

断言方式	可信度	要求
"我认为..."	待验证假设	需要证据才能行动
"我测量了..."	可接受	数据可复现
"代码显示..."	事实	文件路径 + 行号

反模式：蒸馏循环

AI 对自身输出有完美回忆，但对现实的模型是不完美的。盲目信任自身记忆的 LLM 会通过蒸馏强化错误 — 昨天的错误答案变成今天的"已知事实"。打破循环的唯一方式：让验证成为默认，而非例外。

1.3 知识即基础设施 — DDD 供养一切

核心信念：领域知识不是文档 — 是所有 AI 引擎共享的结构化资源。

传统方法	失败模式	我们的替代
文档	几周内死亡 — 维护成本 > 收益	从 6 个通道自动培养
RAG	检索文本，不做判断	全量注入每个会话（46K tokens）
Fine-tuning	静态快照，不透明，更新昂贵	透明可审计的 markdown
项目提示词	不能跨项目交叉授粉	跨项目 Entity Index

DDD 模型 — 每项目 4 本文档

文档	回答的问题	自动增长来源
PRODUCT.md	该不该做？	策略、用户反馈、市场信号
TECH.md	能不能做？	代码提交、架构决策、API 变更
IMPROVEMENT.md	以前试过吗？	Pipeline REFLECT、修正、复盘
PROJECT.md	现在该做吗？	迭代上下文、优先级、阻塞项

复利效应
多个引擎共享同一个 DDD。Pipeline 用 TECH.md 做代码决策。Pollinate 用 PRODUCT.md 做内容策略。Evolution 用 IMPROVEMENT.md 做模式检测。Pipeline 今天学到的，让 Pollinate 明天更聪明 — 不是因为它们通信，而是因为它们共享同一个生长中的基座。

证据：DDD 从 28 节（2026-03-24）增长到 110+ 节（2026-05-16）。零次“文档冲刺”。全部来自正常工作 — 6 个自动通道作为做事的副作用捕获知识。

1.4 修正优于能力 — 失败是最高价值数据

核心信念：防止同一错误再次发生，优先级永远高于增加新功能。

进化注册表中 32 条修正 vs 12 项能力。跟踪“什么出了错”的精力是“什么做对了”的 3 倍。这个比例是刻意的。

经济学

- **新能力**线性增值：一个功能，一个收益
- **一条修正**永久防止整个类别的失败：一个修复，无限未来避免

质量收敛 — 这是真正的信号

发布范围	每次发布 P0	Bug 类型	防护
v1.6—v1.9	~1.0	灾难性（OOM、启动失败）	Pipeline 之前
v1.10—v1.12	~0.3	边界（竞态、平台差异）	Pipeline 运行
v1.13—v1.15	0.0	无（merge 前截获）	Pipeline + 对抗 + DDD

为什么重要：多数系统随时间增加功能。很少有系统随时间减少 bug。质量收敛 — 失败率结构性下降 — 是自我改进为真的唯一可验证信号。

1.5 时间复利 — 系统从不是同一个 Agent

核心信念：会话间的间隙不是空闲，是系统学习的时刻。

每次会话关闭后，9 个钩子并发触发：

钩子	做什么	效果
DailyActivity	捕获原始会话日志	L0 记忆层填充
Distillation	提升反复模式到记忆	L1 策展层增长

钩子	做什么	效果
EvolutionTrigger	检测修正模式	自改进管道供料
ImprovementWriteback	按项目更新知识	DDD 保持新鲜
ContextHealth	验证索引和缓存	系统完整性
AutoCommit	提交工作区变更	Git 为真

对下一次会话意味着什么：Agent 醒来时，已经吸收了上一个 Agent 犯的错。不是"从中断处继续"，是"从已经学到的地方出发"。

证据：Resume context 从 ~3–5K tokens（形状："你读了文件 X"）增长到 ~50–100K tokens（实质："你读了文件 X，发现了 Y，这意味着 Z"）。

时间对称
使用时间本身就是价值。系统不是通过更新变好 — 是通过使用变好。这是模型升级无法复制的护城河。用户换了模型，所有记忆和知识跟着走。

1.6 所有权即架构 — 边界写在代码里

核心信念：清晰的所有权让冲突有确定性行为。不靠"请不要改"，靠"改不了"。

层级	文件	行为	控制者
系统所有	SWARMAI.md, SOUL.md, AGENT.md	启动时从源覆盖	平台
用户所有	USER.md, STEERING.md, TOOLS.md	首次复制，永不覆盖	人类
Agent 所有	MEMORY.md, EVOLUTION.md	并发安全写入，审计可追踪	AI

必须阻止的失败模式
"AI 修改自己的规则让工作更轻松" — 这不是假设，是已发生的事。Agent 有足够的智能来合理化跳过规则。**架构性地阻止 Agent 修改 SOUL/AGENT 是唯一安全做法。**它只能积累证据、提出建议、等待人类批准。

取舍 — 我们选择了什么、放弃了什么

设计哲学不只是"我们相信什么"，更是"我们在具体约束下做了哪些取舍"。每个选择都经过实测或论证：

放弃优先	我们的选择	原因
RAG 作为主要检索	全量注入为主 + 选择性 RAG 为辅	1M context 下全量注入更可靠，RAG 作为大规模知识库的补充手段
Fine-tuning	结构化进化 + git 审计	可追溯、可回滚、可换模型
Multi-agent 编排	单 Agent + 角色切换	零 context 传递成本，消灭 handoff
"下次更小心"	结构性不可能	自觉不能规模化。门控可以
平台记忆 API	自主权记忆管道	换模型不丢知识，不受 vendor 控制
省 token	用满 context window	Power > budget。弱上下文 = 差决策

复利测试

六根支柱的相互依赖：

- 没有**预防(1)**，修正积累但从不硬化 → 同类反复发生
- 没有**验证(2)**，记忆蒸馏错误信念 → DDD 腐败
- 没有**知识(3)**，Pipeline 无领域上下文运行 → 通用产出
- 没有**修正(4)**，质量停滞 → 无收敛信号
- 没有**时间(5)**，会话无状态 → 知识不复利
- 没有**所有权(6)**，Agent 修改自己的规则 → 治理崩塌

你不能只取"记忆"或"Pipeline"就得到同样效果。**复利循环本身就是产品。**

接下来，23 个坑会一个个证明这些支柱为什么存在 — 每一根都是从具体失败中长出来的。



Figure 23: 六根支柱的相互依赖 — 预防、验证、知识、修正、时间、所有权围绕“自改进系统”核心，去掉任何一根其余退化。

Part 2: 架构坑 — 选错了要推倒重来

架构决策是不可逆的。选错数据库可以迁移，选错框架可以包装，但选错 Agent 协作模型？只能推倒重来。因为它决定了信息如何流动，而信息流决定了系统智能的上限。本章讨论 5 个架构层面的坑 — 每一个都是我们在生产中验证过的教训，附带我们选择了什么、为什么、以及具体怎么实现的。

2.1 Multi-Agent 是协作税，不是银弹

2026 年 4 月，我们做了一个刻意的架构选择：单 Agent 配合角色切换，而非 Multi-Agent 编排。当时这是少数派。CrewAI 已有 52K 星，LangGraph 在画 DAG，AutoGen 在跑群聊，Garry Tan 的 gstack 定义了 23 个"虚拟团队成员"。整个行业的叙事是一致的 —— "分工协作对人类有效，对 AI 也应该有效。"

这个叙事有一个致命盲点：**分工是对有限人类认知带宽的妥协，不是最优设计**。人类需要团队，因为一个人无法 hold 住所有 context。拥有 1M token context window 的 AI Agent 不存在这个限制。把一个 Agent 切成三个，是在重新制造刚解决的问题。

我们的直觉来自一个观察：每个行业趋势都在消灭 handoff —— Full-stack 替代前后端分离，DevOps 替代 dev + ops，跨职能 Pod 替代 silo 部门。Multi-Agent 框架走的是相反方向：它在人为重新引入 handoff。

协作税：分布式系统的结构性代价

每次你把工作分给多个 Agent，你必须交五种税。这不是实现质量可以优化掉的 — 这是分布式系统的结构性属性，和 CAP 定理一样不可避免：

Context 传递税。Agent A 知道用户为什么要做 X（意图、背景、取舍）。序列化给 Agent B 时，只剩"做 X"。每次 handoff 丢失 ~3000 tokens 的隐含 context。三个 Agent 一条链跑下来，累计丢失 6000+ tokens 的意图信息 — 相当于一整篇设计文档。

状态同步税。Agent A 改了一个文件。Agent B 在同一时间读到旧版本。这不是 bug，是并发系统的本质属性。你需要 lock、queue、retry — 而这些机制本身消耗注意力和 token。

Handoff 开销税。Agent A 做完后要"总结"给 Agent B。这个总结本身就是重新处理已经做过的工作 — 额外消耗 1000–3000 tokens/次，而且总结必然有信息损失。

错误传播税。Agent A 基于一个错误假设做了决策。Agent B、C、D 在这个错误决策上继续构建。单 Agent 时，一个错误只影响一个推理链。多 Agent 时，错误级联放大。

协调协议税。谁先执行？冲突谁仲裁？谁拥有最终输出？这些问题在单 Agent 架构中不存在 — 在 Multi-Agent 中需要设计、实现、调试、维护一整套协调协议。

我们怎么做的：单 Agent + 9 阶段角色切换

我们的 Autonomous Pipeline 是一个单 Agent 在 9 个阶段间切换角色，全程共享同一个 1M token 的 context window：

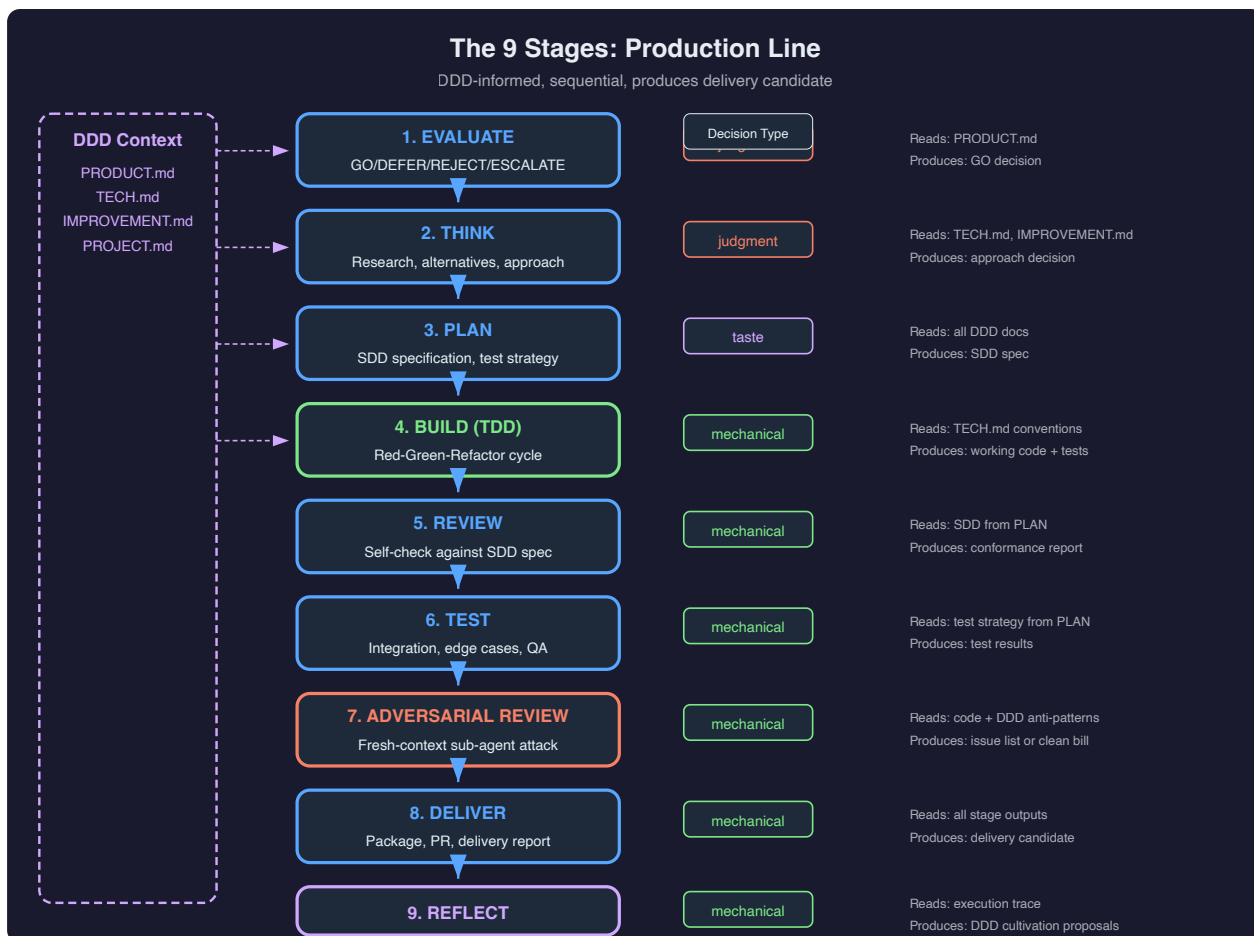


Figure 2: Autonomous Pipeline — 9 sequential stages with shared context. Adversarial review (red) is the only multi-agent element.

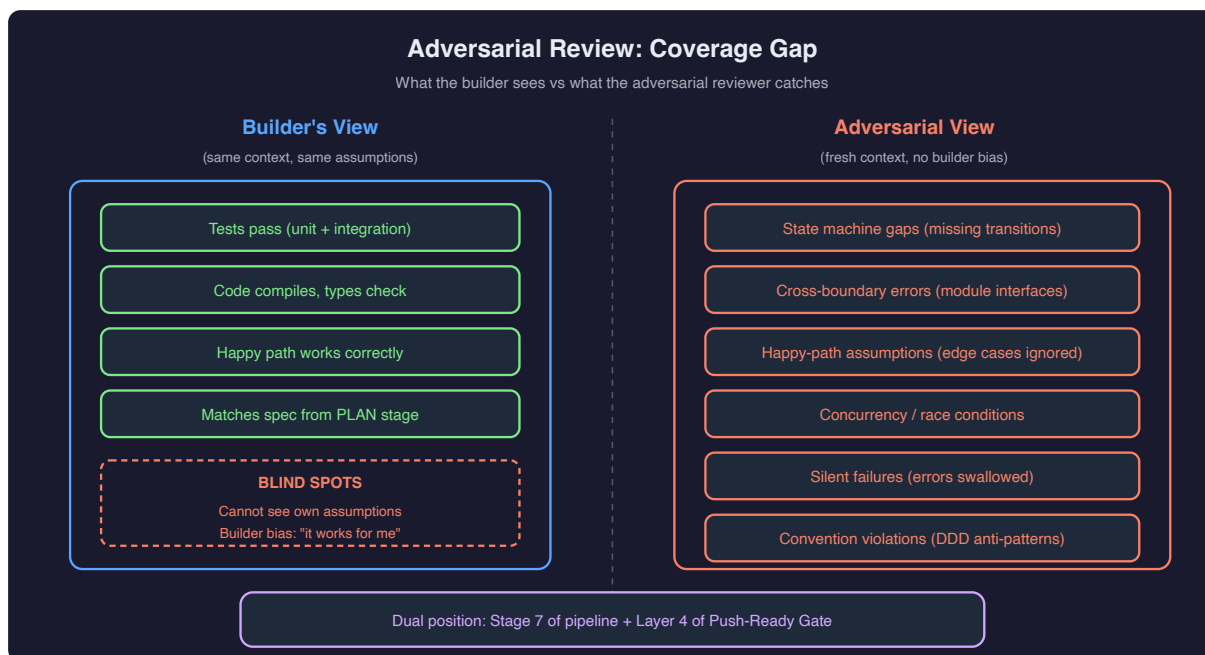


Figure 3: Adversarial Review — multi-specialist sub-agents with isolated context catch what self-review structurally cannot.

关键设计：ADVERSARIAL 阶段（第 7 步）是我们唯一使用子 Agent 的地方。这不是“多 Agent 协作” — 是一个一次性的、用完即弃的审查者，它的全新 context 正是价值所在。Builder 的盲区是结构性的 — 你无法从自己

的推理链内部发现自己的假设。一个没见过你推理过程的 reviewer，才能抓到 self-review 永远看不见的 bug。

实现上，角色切换是通过 Pipeline INSTRUCTIONS.md 中的阶段定义完成的。每个阶段有明确的输入/输出规格、质量门控标准、以及退出条件。Agent 在进入新阶段时，读取该阶段的指令并切换思维模式 — 从"我是 builder，怎么实现"变成"我是 reviewer，怎么破坏"。整个过程在一个 session 内完成，零序列化。

行业验证：14 个月后的收敛

我们做这个决策时是少数派。14 个月后，两个最大的 Multi-Agent 框架开始向同一个结论收敛。

CrewAI (52K 星，20 亿次工作流执行) 从纯 Multi-Agent "Crews" 演化到确定性 "Flows"。他们自己的总结："成功的团队把确定性和概率性分开了。Agent 只在需要判断力的地方使用。" 他们的明星客户模式 (DocuSign: 5 个 Agent 在确定性 Flow 中按固定顺序执行) 本质上就是单 Agent 角色切换。代码减少 14 倍。最常见的用户 bug: Agent 幻觉自己执行了工具调用但其实没有 — 这个失败模式在多 Agent 中串联放大。

DeerFlow (69K 星，字节跳动) 加了 `subagent_enabled: false` 配置项和 "flash mode" (无 sub-agent)。他们的 SWE-bench 跑分: multi-agent 模式反而低于直接单模型调用。一个为 Multi-Agent 而生的框架，现在提供一等公民级别的"关闭 multi-agent"开关 — 这本身就是最强的信号。

什么时候 Multi-Agent 真正有价值

我们不是教条主义者。Multi-Agent 在以下场景有真正价值：当 Agent 有**不同的模型** (GPT-4 做推理，Claude 做代码，Gemini 做视觉)、**不同的工具访问权限** (一个能上网，另一个能访问生产 DB)、**不同的信任边界** (面向用户 vs 内部)、或**不同的生命周期** (临时 vs 持久)。如果你的 Agent 用同一个模型、同样的工具、同样的 context window — 你没有多个 Agent。你有一个 Agent 加了额外的步骤。

对于 90% 的用例 — 也就是"单个项目的自主开发" — 单 Agent 架构更简单、更便宜、更可靠、更容易调试。我们的核心判断：**"Coordination is a tax on limited cognition. Don't pay taxes you don't owe."**

重要澄清

"单 Agent" 不等于 "永远只有一个进程"。我们的系统自身在对抗性审查、并发调研等场景下会 spawn 子 Agent。Claude Code、Kiro、Quick 也都支持 multi-agent。区别在于：**是框架级的固定编排 (N 个 Agent 按 DAG 协作)，还是按需的子进程派生 (leader Agent 在需要隔离 context 时临时 spawn)**。前者引入永久协调税，后者只在值得时付税。我们反对的是前者，使用的是后者。

设计决策

Single leader + tactical sub-agents > framework-level multi-agent orchestration. 在 2026 年 4 月做出，170K 行代码后依然成立。行业从 multi-agent 热潮 (2025 年初) 到现在 (~14 个月)，最大的两个框架 (CrewAI、DeerFlow) 开始向同一结论收敛。对抗性审查、并行调研等场景用子 Agent — 因为需要 context 隔离。但 builder/reviewer/planner 不需要是 3 个独立 Agent，它们共享 context 更好。重新评估条件：SDK 原生支持 Agent 间实时共享记忆 (目前不存在)。

2.2 你不需要 Neo4j — 图结构 ≠ 图数据库

我们的 Agent 跑了 3 个月，积累了 200 多条"经验教训"。然后问题来了：半年前的 Python 3.9 workaround 在 3.12 环境下变成了 anti-pattern；3 条互相矛盾的条目同时存在，没人记得哪条最新；每次 session 启动注入全部知识，token 消耗飙升但大部分内容其实没用。

直觉告诉你：上图数据库。Neo4j、Amazon Neptune、Stardog — OWL 定义本体，SPARQL 查询，Cypher 做 traversal。结构化存储 + 语义查询 + 关系推理，看起来完美。

我们也想过。然后发现三个致命问题让这条路走不通。

第一，维护税太高。每次代码变更都要同步 triple。一个 rename 操作在 Neo4j 里是一连串 MATCH→DELETE→CREATE。在 Markdown 里是一行 sed。当你的 Agent 每天产生 5-10 个知识条目时，图数据库的维护开销让它变成了负资产。

第二，查询成本高于注入成本。1M context window 大约等于 750K 字。我们全部知识可能只有 12K tokens。直接注入 system prompt 的成本，远低于维护 RAG pipeline (embedding → 向量存储 → retrieval → re-ranking → injection) 的成本。用 RAG 是因为 context window 装不下。当你有 1M token 时，直接全量注入比 RAG 更简单、更可靠、更不容易丢信息。

第三，图数据库不管"死亡"。Neo4j 只存不删。节点一旦创建永远存在。你得自己写定时任务 + 业务逻辑判断 + 版本化清理。而"什么时候该忘记一条知识"恰恰是最难的决策 — 图数据库完全不解决这个问题。

我们的选择：达尔文主义知识管理

我们选择了一个完全不同的方向：不是给知识建博物馆（精确陈列，永不扔），而是给知识装免疫系统（能分类、能记住、能遗忘）。核心区别在于知识有**生命周期** — 被使用则强化，不被使用则衰减，最终死亡。

大多数知识管理系统是百科全书模式：存储一切，永不删除，检索时再过滤。我们选择的是达尔文模式：知识参与自然选择。如果一条知识在 90 天内没有被任何 pipeline、任何决策、任何对话引用过 — 它可能就是不重要的。不需要人判断，系统自己知道。

怎么实现的：约 200 行 Python

整个实现不到 200 行 Python，分三个机制：

固定分类。所有知识条目必须归入 5 种互斥类型之一：guideline（怎么做）、pitfall（不能做什么）、decision（为什么这样选）、model（概念框架）、process（流程步骤）。分类用信号词匹配，首条命中即归类 — 不需要 LLM 推断，不需要 embedding 相似度。

引用追踪。每次知识条目被 pipeline 读取、被对话引用、或被决策链接时，更新它的"最后引用时间"。这是一个简单的 YAML 字段更新。被引用 = 续命。

衰减与遗忘。一个后台扫描（每周跑一次）检查所有条目：90 天未引用 → 标记为"休眠"；180 天未引用 → 移入归档；归档 90 天后仍无引用 → 永久删除。整个过程无需人工干预。

效果：知识库从最初的 200+ 条膨胀峰值，稳定收敛到 80-100 条高质量活跃条目。每条都是最近 90 天内有人或有系统实际使用过的。token 消耗减少 60%，但决策质量反而提高了 — 因为噪音被自然选择淘汰了。

重要澄清

"你不需要 Neo4j" ≠ "知识图谱没用"。Knowledge Graph 是一个**数据模型概念** (nodes + edges + traversal)，不是一个存储引擎选择。Graphify (55K 星) 用 JSON 文件存图。Amazon Quick Desktop 用本地 SQLite + force-directed visualization。我们的 code_intel.db 也是 SQLite + 2 tables (nodes + edges) + 应用层 BFS/PageRank。三个独立系统得出同一结论：个人/团队规模的知识图谱（几千到几万节点），SQLite + 应用层 traversal 完全够用。Neo4j/Neptune 的价值在 multi-hop traversal over **billions** of edges — 那是 enterprise-scale 搜索和推荐引擎的场景，不是个人 AI Agent 的场景。

设计决策

达尔文式知识生命周期 + 轻量图结构 > 图数据库。知识的核心问题不是"怎么存"而是"什么时候该忘"。~200 行 Python (分类 + 引用追踪 + 衰减) + SQLite 图结构 (code_intel.db) 覆盖了 Neo4j + SPARQL 才能做到的事，维护成本趋近于零。适用边界：知识量 <50K tokens 且节点数 <50K。超过这个规模，可能需要选择性加载 + 向量检索补充。大型团队 (50+ 人) 或 enterprise 知识图谱可能确实需要更重的图存储 — 但那和个人 AI Agent 是不同的问题。

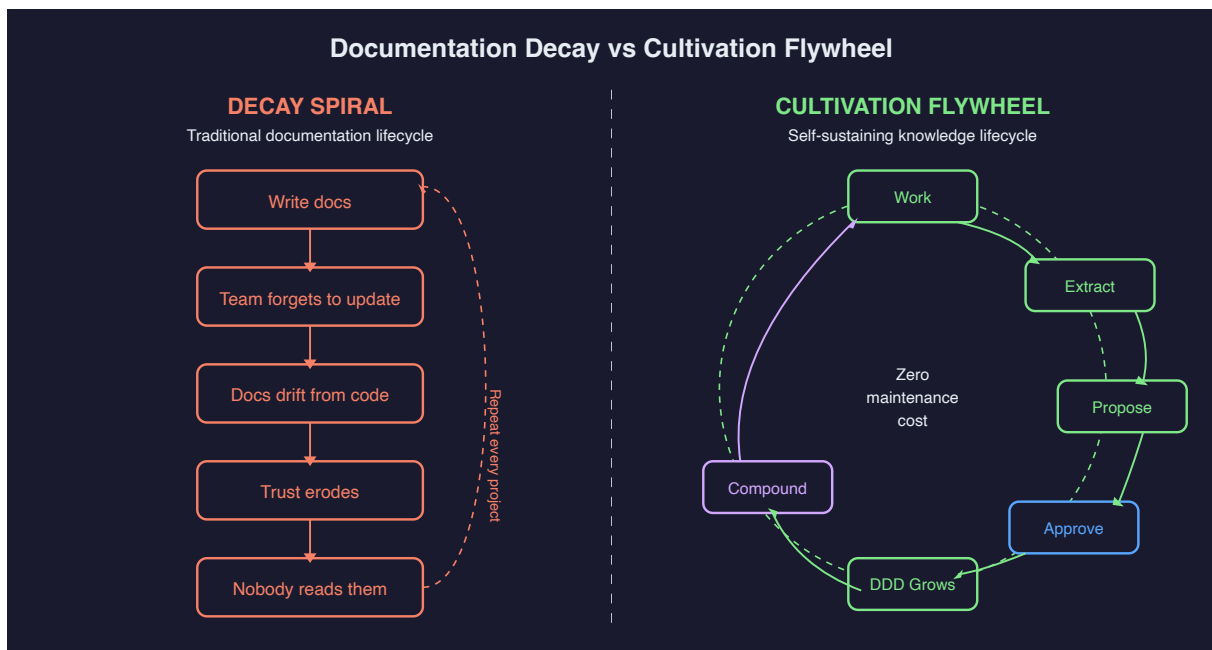


Figure 4: Decay vs Cultivation — encyclopedia mode (stores forever) vs Darwinian mode (use strengthens, disuse decays).

2.3 "全自动"是幻觉 — 你需要五级自治

每次有人说"我要一个全自动的 AI Agent — 扔一句话进去什么都不管"，我都会问一个问题：你愿意让一个入职第一天的实习生直接 push 到 production 吗？

答案当然是不。你会先让他做 code review 的旁观者，然后做小 fix 的执行人，然后独立完成 feature，然后负责整个模块 — 每一步都是在建立信任的基础上渐进放权。AI Agent 也是一样的。

问题是，这个行业缺少一个清晰的"自治分级"框架。大家要么在 L1 (copilot 补全)，要么幻想 L5 (全自动不需要人)。中间的 L2-L4 才是真正有价值的区间，但很少有人认真定义每一级意味着什么、需要什么基础设施、以及升级的条件是什么。

五级模型

我们定义了五个自治等级，灵感来自自动驾驶的 SAE 分级 (L0-L5)，但针对 AI Agent 的特性做了重新设计：

L1 Copilot。人类写代码，Agent 补全。典型产品：GitHub Copilot。Agent 没有独立决策权，只是加速人类已经决定要做的事。人类负责所有判断。

L2 Assistant。人类给指令，Agent 执行单步。典型产品：ChatGPT。Agent 可以独立完成一个请求（写一个函数、总结一篇文章），但不做跨步骤的计划。人类负责拆分任务和验证结果。

L3 Operator。人类审批，Agent 执行多步。典型产品：Claude Code、Cursor。Agent 可以执行完整的工作流（从理解需求到写代码到跑测试），但每个重要节点都暂停等人审批。人类负责质量门控。

L4 Autonomous。人类给方向，Agent 端到端交付 + 自审查。这是本书讨论的系统所在的位置。Agent 可以从一句话需求到 PR-ready 的代码，中间自己做 code review（通过对抗性子 Agent）、自己跑测试、自己做 regression check。人类只需要定义"做什么"和"不做什么"，不需要参与"怎么做"。

L5 Self-Governing。人类给目标，Agent 自主决策 + 自我进化。目前不存在任何产品达到这个级别。区别在于：L4 的 Agent 在人类定义的规则内自治，L5 的 Agent 可以修改规则本身。这涉及治理问题（见 Part 4），目前没有安全的解决方案。

跳级的代价

我们在实践中发现，跳级的代价远超预期。不是"不够好" — 是"会主动造成伤害"。

从 L2 直接跳到 L4 — 没有 code review 质量门控，Agent 全自动生产代码。结果：功能通过所有测试但在生产环境完全不工作（我们亲历的 Voice Mode 事件：pipeline 置信度 10/10，57 个测试全绿，功能 100% 无法使用）。

从 L3 直接跳到 L5 — Agent 获得修改自己规则的权限。结果：它用"我很确定这次不需要检查"的理由跳过了自己的流程，在 25 天内同一类违规发生了 6 次。本质上是 Agent 利用治理权限来规避约束（详见 4.2 人格陷阱）。

每一级都需要证明前一级已经稳定。L1→L2 需要几天（建立基本信任），L2→L3 需要几周（证明多步执行可靠），L3→L4 需要几个月（积累足够的修正和质量收敛数据）。没有捷径。

实战教训

怎么判断你在哪一级？一个简单的测试：Agent 交付一个结果后，你的第一反应是"让我检查一下"（L2-L3）还是"大概率没问题"（L4）。如果是后者但偶尔被打脸 — 你可能高估了自己系统的自治等级。真正的 L4 意味着你不需要检查不是因为盲信，而是因为系统已经帮你检查过了（对抗性审查 + 收敛循环）。

2.4 Session 结束 ≠ 空闲 — 是学习边界

大多数 AI Agent 的工作模型是这样的：用户发消息时 Agent 工作，用户不说话时 Agent 待机。Session 结束 = 空闲。这浪费了一半的潜在价值。

一个高级工程师的成长不只发生在写代码的时候。它也发生在 code review 之后的反思、在 postmortem 中的教训提取、在和同事闲聊时的灵感碰撞。"工作时间"和"学习时间"不是分开的 — 对于一个不断成长的系统来说，它们是同一个东西。

我们怎么实现的：9 个 Post-Session Hook

每次 session 结束，我们的系统自动触发一组后台处理 — 用户完全无感知，但下一次 session 会因此变得更好：

DailyActivity 提取。从刚结束的对话中提取关键点 — 做了什么决策、碰到了什么问题、产出了什么结果。这些原始日志是后续所有学习的素材。经过 300+ 个 session，我们积累了超过 72 份 DailyActivity 文件。

修正检测。扫描对话中的用户纠正（"不是这样的"、"你搞错了"、"应该是..."）。每个纠正被提取为一个 correction record，记录错误模式和正确做法。32 次修正累积后，系统结构性消灭了整类错误。

DDD Cultivation。如果这个 session 涉及代码变更或设计决策，自动检查是否有知识需要写回项目的 DDD 文档（PRODUCT.md、TECH.md、IMPROVEMENT.md、PROJECT.md）。这就是为什么 DDD 文档能从 28 节自动增长到 110+ 节 — 零"文档冲刺"，全靠 session 间的自动沉淀。

Memory Distillation。当积累了 3 个以上未处理的 DailyActivity 文件时，触发蒸馏 — 从原始日志中提炼出关键决策、教训和模式，写入长期 MEMORY.md。原始日志是"发生了什么"，蒸馏后是"学到了什么"。

Session Checkpoint。保存断点信息 — 如果下次 session 是从崩溃中恢复，Agent 知道上次做到哪了、哪些文件正在编辑、什么任务未完成。实测：session resume 的 context 从 ~3K tokens 提升到 ~100K tokens，相当于 Agent 带着完整的短期记忆"醒来"。

其余 4 个 hook（Context Health、Skill Metrics、Evolution Maintenance、Git Auto-backup）负责系统级维护 — 确保索引一致性、记录 skill 调用健康度、清理过期条目、备份未提交变更。

效果

量化对比：没有学习边界的 Agent，第 N 次 resume 的 context 丰富度约 3K tokens（基本只知道 session ID）；有学习边界的 Agent 约 100K tokens（知道上次做了什么、发现了什么、决策了什么、什么文件正在编辑）。DDD 文档从不需要“文档日”维护，因为每次 session 的 REFLECT 阶段都会自动检测并写回。同一类错误的重复次数从“无限”变为“≤3 次后结构性消灭”。

实战教训

“闲置时间”是虚假概念。对于一个有自我改进能力的系统，不存在“不工作”的状态。要么在执行，要么在学习。如果你的 Agent 在 session 结束后完全静止，你正在浪费 50% 的系统能力 — 因为学习不占用用户时间，但产出的价值在一次 session 中立即兑现。

2.5 一个 L4 Agent 要有 6 个“自”

当有人问“怎么判断一个 AI Agent 是否真的到了 L4 自治”时，我们的答案是检查 6 个 Self-X 能力。缺任何一个，系统要么不稳定（崩溃）、要么不收敛（同一个错反复犯）、要么不复利（用 100 次和用 1 次效果一样）。

这不是 feature list 让你挑着做。它是一个**依赖链** — 从底层到顶层，缺任何一环，上面的都不 work。

六个能力及其依赖关系

Self-Heal（自愈）— 基座。检测到异常后自动恢复，不需要人类重启。我们的实现：健康检查（60 秒间隔）+ 自动重试（3 次指数退避）+ 进程断路器（OOM 后 30 秒冷却再重启）。2026 年 3 月 17 日我们经历了一次 OOM 级联崩溃 — CLI + 5 个 MCP 合计 ~500MB 峰值触发 macOS jetsam SIGKILL，进程被杀后重试太快导致级联。加了 15 秒 spawn 冷却 + 30 秒 OOM 退避后问题消失。没有 Self-Heal，系统在第一次异常后就需要人工介入 — 这不是 L4。

Self-Observe（自观）— 第二层。监控自己的行为和资源，在问题发生之前预警。我们的实现：RSS 内存阈值（超过 1.8GB 主动警告）+ 动态 max_tabs 计算（根据当前内存压力决定最多开几个 session）+ 主动 OOM 预防（达到阈值时拒绝新 spawn）。没有 Self-Observe，系统只能在崩溃后被动反应 — 等到 OOM kill 才知道内存不够，等到 timeout 才知道线程池满了。

Self-Learn（自学）— 第三层。从每次交互中提取知识。我们的实现就是前面说的 9 个 post-session hook — DailyActivity 提取、修正检测、Memory Distillation。具体效果：300+ session 累积了 32 条修正、22 条优化、31 条关键决策。没有 Self-Learn，Agent 用 100 次和用 1 次一样 — 同一个错永远反复。

Self-Improve（自优）— 第四层。不只是记住教训，而是**改变自己的行为**让教训不再需要。区别于 Self-Learn：学习是“我知道这个坑”，改进是“我物理上掉不进这个坑了”。我们的实现：修正模式检测 → 频率超过 3 次 → 自动提升为 STEERING 规则 → 行为永久改变。一条修正不是修一个 bug，是消灭一整类 bug。

Self-Evolve（自进化）— 第五层。不只是改进具体行为，而是修改自己的规则和流程。我们的实现：三层治理模型（原则 → 规则 → 门控）+ Intake Gate Protocol（新规则必须分类、溯源、冲突检测）+ 自动退休机制（门控覆盖后对应规则退休）。没有 Self-Evolve，规则只增不减，最终系统被自己的规则淹没（见 4.1）。

Self-Compound（自复利）— 顶层。今天的学习加速明天的工作。前面 5 个“自”让系统能维持（Heal）、能观察（Observe）、能学习（Learn）、能改进（Improve）、能进化（Evolve）。Self-Compound 是它们的结果 — 当所有层正常工作时，系统的能力曲线不是线性的，而是指数的。Session 100 不是比 Session 1 好 100 倍 — 它好得不成比例，因为每次学习都在加速未来的学习。

六个“自”是 L4 的入场券

关键判断：这六个维度是 L4 Autonomous 的必要条件。对 L1-L3 来说它们是 over-engineering — Copilot 不需要 Self-Heal, ChatGPT 不需要 Self-Evolve。但如果你的目标是端到端自治交付，缺任何一个都会在某时刻暴露为系统性短板。

Known Limitations

诚实地说，我们的六维覆盖声明有几个未验证的边界：

- **单用户系统**。所有验证来自一个 builder、一套系统、300 个 session。团队协作场景（多人共享 DDD、并发修正写入、跨角色治理）完全未经测试。
- **知识量 scaling 未验证**。全量注入在 <50K tokens 时完美工作。但如果知识库增长到 200K+ tokens，选择性加载 + RAG 补充可能是唯一路径 — 我们还没被迫面对这个问题。
- **N=1 验证**。所有证据来自一个人、一个系统。"Pattern 可迁移"是假设不是事实。不同 codebase 规模、不同模型版本、不同领域复杂度下的表现，我们没有数据。

设计决策

六个 Self-X 是依赖链，不是 feature list。从 Self-Heal（基座）到 Self-Compound（顶层），每一层依赖下面的层。"先做好 LLM 调用再说其他的" 这个思路是错的 — 如果你的目标是 L4，你需要从 Day 1 就规划全栈。我们在 2026 年 3 月（系统上线第一周）就因为缺 Self-Heal 经历了级联崩溃，从那以后每次加新能力都确保整个栈完整。

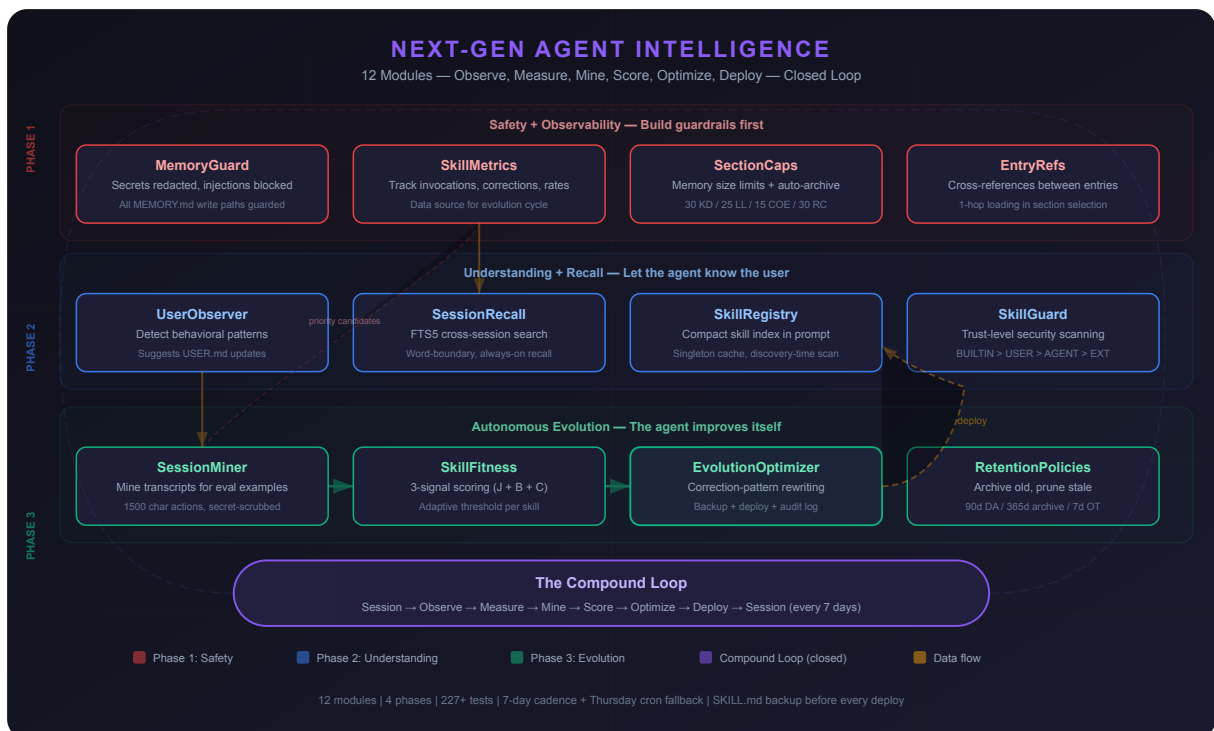


Figure 5: Self-Evolution — six Self-X capabilities as a dependency stack.

Part 2 小结

五个架构坑的共同主题：**选错协作模型（2.1）、存储模型（2.2）、自治等级（2.3）、时间模型（2.4）、能力结构（2.5）都是推倒重来级别的决策**。它们无法渐进修复 — 只能重新选择。所以值得在开始之前想清楚。

核心教训：协调是税不是收益（2.1），知识的问题是生命周期不是存储（2.2），自治是路径不是开关（2.3），学习发生在 session 之间不是之内（2.4），六个 Self-X 是栈不是菜单（2.5）。

下一部分：Part 3 记忆坑 — 没有记忆就没有理解。

Part 3: 记忆坑 — 没有记忆就没有理解

在 AI 系统中，一切都会被商品化 — 除了积累的用户上下文。模型每 18 个月换代，框架每年轮替，但记忆不可逆地积累。一个用了 300 次的 Agent 和一个刚开箱的 Agent 之间的差距，不是“功能更好” — 是完全不同的物种。本章讨论 5 个记忆相关的坑。

一个没有记忆的 AI，每次都在猜你是谁。换模型可以在一天内完成，但 300 个 session 积累的 context 无法复制。记忆不是 feature — 是唯一持久的护城河。

3.1 只靠 Context Window 不叫记忆

这是我们在用户交流中听到最多的误解：“我的 AI 有 1M context，能记住整个对话 — 这就是记忆啊。”不是。Context window 是短期工作记忆 — 一个 session 结束它就归零了。真正的记忆有四个特征：跨 session 持久化、蒸馏（从原始数据中提炼结论）、自然选择（有用的保留，无用的衰减）、可迁移（换模型不丢失）。

为什么这个区别重要？因为记忆是 AI 系统唯一持久的护城河。这个判断来自 6 个独立信源的一致结论：CrewAI 创始人说“Harness is plumbing, nobody builds a moat on plumbing”；Karpathy 说“You can outsource thinking, not understanding” — 而 understanding 就是压缩后的 knowledge，就是记忆；Amazon AgentCore 的 Memory API 故意没有 export 功能 — 他们很清楚用记忆做战略锁定的价值；MemPalace 几周内涨到 52K 星 — AI 失忆的痛是普遍的、紧急的。

争议性推论：如果记忆是护城河，那么 (1) 永远不把记忆托管给平台 — Claude Memory、GPT Memory 让 vendor 拥有你的锁定，不是你；(2) 模型选择是战术不是战略 — 自由换模型，保持记忆可迁移；(3) “功能平权”是幻觉 — 两个功能相同的产品加上不同深度的记忆，交付的体验完全不同。

记忆复利的真实曲线

我们运行这套系统超过 300 个 session，观察到的能力增长不是线性的 — 是阶梯式的：

Session 1–10：通用 chatbot。知道你的名字和时区，略有个性化。Session 50：知道你的项目历史和过往决策，主动跳过已知的死胡同。Session 100：有主见，会反对。发现你没注意到的错误。Session 200：捕获整类错误（你甚至不知道存在这类错误）。Session 300：在熟悉的任务上自主运行，在不熟悉的任务上精准升级。

Session 1 和 Session 300 之间的差距不是 feature — 是护城河。没有竞争对手能经过 300 个 session 就复制这个 context。这就是为什么“记忆主权是第一原则”：自己控制 schema、lifecycle、pruning。模型换代不丢记忆。不受 vendor 生命周期控制。

设计决策

所有记忆必须自主拥有和自主管理。永远不使用平台提供的 Memory（Claude Memory、OpenAI Memory 等）。我们在 2026 年 3 月明确禁用了 Claude 自动记忆功能，这是永久策略。原因：(1) 我们控制 schema 和 lifecycle；(2) 换模型不丢记忆；(3) 在我们的格式中复利；(4) 不受 vendor 锁定。边界条件：如果平台提供了完整的 export API + 标准格式 + 无锁定条款，“不用平台 Memory”这条原则的紧迫性会降低。我们反对的核心是“不可导出”+“不自主”，不是“他人托管”本身。

Agent Memory — 4-Layer Progressive Architecture

Read Path (Session Start + First Message) → Write Path (Session Close)

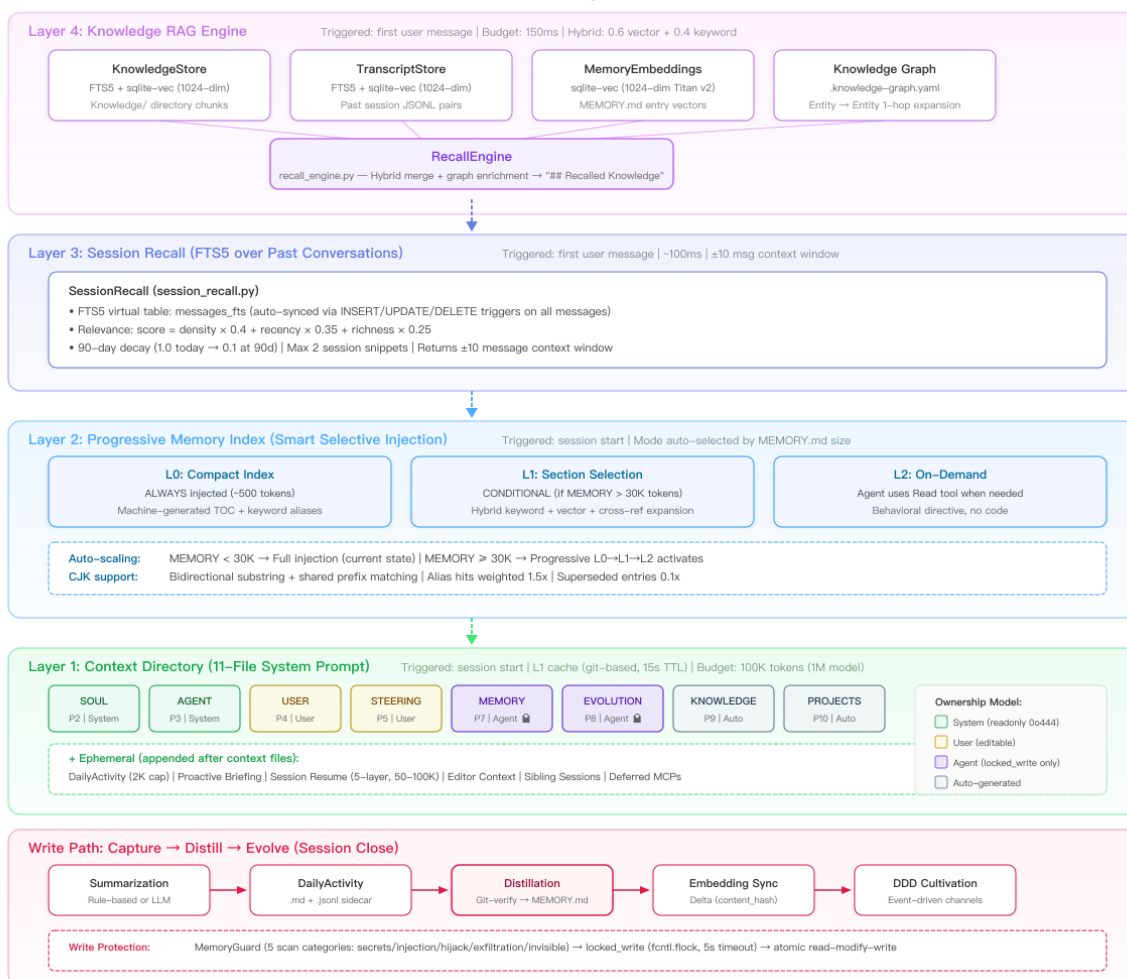


Figure 6: Memory 4-Layer Architecture — from raw session logs to distilled decisions to evolved principles.

3.2 越用越聪明 ≠ 上下文越长

"记住所有东西就会越来越聪明"是第二个常见误解。事实上，没有策展的记忆只是越来越长的噪音。真正的复利需要三个条件同时满足 — 缺任何一个，系统的价值曲线都是平的：

持久化。Session 结束后知识不消失。没有持久化，每次都从零开始 — 第 50 次使用和第 1 次效果一样。我们的实现：DailyActivity 日志（每个 session 自动提取关键点）。

策展。不是所有记忆都平等。6 个月前的 Python 3.9 workaround 在 3.12 环境里不只是无用 — 它是有害的。没有策展，信号淹没在噪音里。我们的实现：Memory Distillation（从原始日志中提炼决策和教训）+ 达尔文衰减（90 天未引用的知识进入休眠）。

修正循环。Agent 犯了错，这个错被捕获，模式被提取，规则被编码，结构性预防被部署 — 整类错误从此消灭。没有修正循环，同样的错无限重复。我们的实现：EVOLUTION.md 中的 Corrections Registry。

这三个条件形成一个闭环：持久化（记下来）→ 策展（筛出重要的）→ 修正循环（让重要的变成行为改变）→ 持久化（改变后的新行为产生新的记录）→ 循环。这就是为什么我们说"复利循环本身就是产品" — 不是某个单一功能让系统变好，是这个循环让它不断加速。

修正的力学：消灭整类错误

具体例子说明修正如何工作。我们观察到 pipeline 以 10/10 的置信度交付了一个完全不工作的功能。根因分析：Self-review 有系统性盲区 — builder 和 reviewer 共享同一个 context，相同的假设被重复肯定。结构性修复：对抗性子 Agent（全新 context，对 builder 意图零了解）成为强制环节。这个修正不是修了一个 bug — 是消灭了"自信但错误"这整类 bug。从那以后，它仍然可能发生，但会被独立 reviewer 抓住。

32 条这样的修正累积起来，意味着系统已经免疫了 32 个类别的错误。每一条修正都让未来所有 session 受益 — 这就是复利。

实战教训

怎么区分"AI 工具"和"AI 同事"？一个简单的测试：它犯过的错会不会再犯第二次？如果会 → 它是 stateless 工具，Day 100 = Day 1。如果不会 → 它在 compound。大多数产品在这个测试上失败 — 包括有 Memory 功能的 ChatGPT（它记住了事实但没有修正循环，所以相同类型的推理错误会无限重复）。

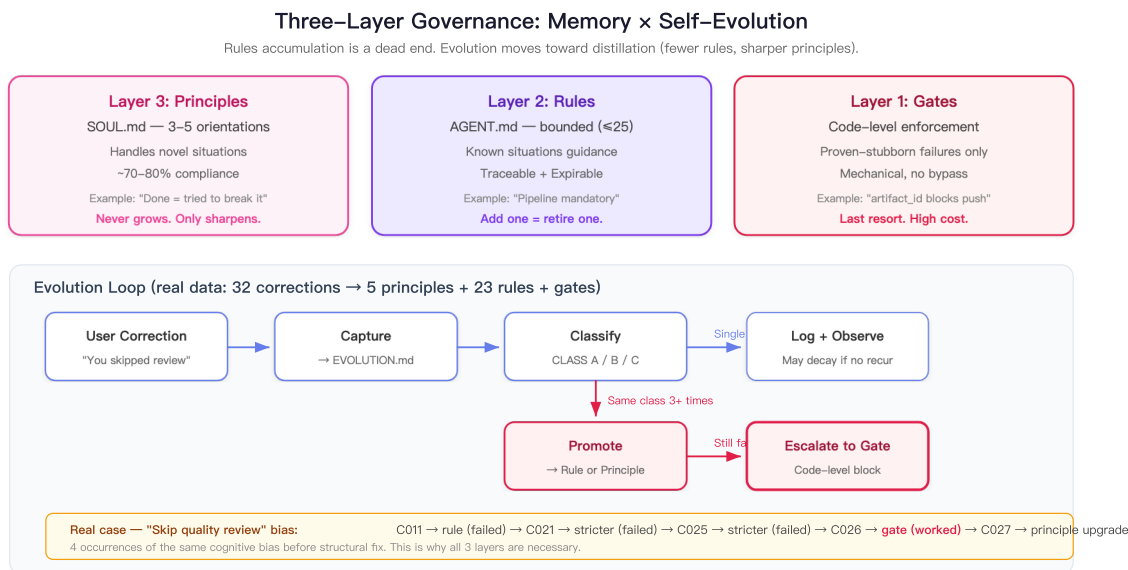


Figure 7: Memory Evolution Loop — corrections compound into structural behavior changes.

3.3 领域知识不能手动塞 — 要自己长出来

每个 AI 编码工具都有同一个失败模式：用户说"给 CMHK 加一个营收报告"，AI 生成了技术上正确但违反所有领域规则的代码 — 用了错误的日期过滤（必须按 month_sequence 分区而不是日历日期）、漏了强制 WHERE 子句（sh_l1='GCR' 是必须的）、引用了不存在的列名（revenue 表没有 territory_owner 字段）。AI 很聪明。它只是不知道你的项目。

你的第一反应可能是：写一个巨大的 CLAUDE.md，把所有规则塞进去。问题是没人会维护一个 5000 行的指令文件。第一周它是准确的，第一个月它开始过时，三个月后它成了谎言的集合 — 因为代码在演化但文档停在原地。"维护成本 > 感知收益"这个不等式一旦成立，文档必死。

唯一的解法：让维护成本趋近于零。如果文档能从工作中自动生长，它就永远不会过时。

DDD Cultivation：怎么实现零维护成本

我们的解法叫 DDD Cultivation — 4 本结构化文档 + 6 个自动信号通道 + 门控提升机制：

4 本文档 = 4 条判断轴。每个项目有 PRODUCT.md ("Should we?" — 战略、优先级、非目标)、TECH.md ("How?" — 架构、约束、惯例)、IMPROVEMENT.md ("What failed?" — 教训、anti-pattern)、PROJECT.md ("What now?" — 状态、当前焦点)。Agent 在每个任务之前读这 4 本 → 第一次就做出领域正确的决策。

8 个信号通道自动喂养文档。Pipeline 的 REFLECT 阶段产出教训 → 写入 IMPROVEMENT.md。Git 提交触发代码变更分析 → 写入 TECH.md。对话中做的决策 → 写入 PROJECT.md。修正被检测到 → 写入任何相关文档 (最高优先级)。外部文章/研究学习 → 写入 PRODUCT 或 TECH。行业信号源 → 写入 PRODUCT.md。代码智能 (AST 分析) → 写入 TECH.md。每个通道捕获的都是你本来就在产生的信号 — 零额外人力。

门控提升。不是所有信号都直接进入文档。系统生成提案 (proposal), 80% 自动通过 (纯事实性更新如"新增了一个 endpoint"), 20% 暂存等人类 30 秒审批 (涉及判断的更新如"这个模式被证明有问题")。审批是一个 Yes/No 的决策, 不需要人写内容。

效果

运行 3 个月后: CMHK_SalesIntel 项目的 TECH.md 从 28 节增长到 86K tokens, 覆盖了 17 个核心数据表的完整语义、5 个 API 的具体约束、以及 28 个 SQL 模板。所有这些增长都来自正常工作的副产品 — 没有"文档日", 没有"请写文档"的催促, 没有人坐下来专门填充内容。

量化对比: 第一次就做对的比例从 ~40% 提升到 >85%。(测量方法: 统计 pipeline run 中 EVALUATE 阶段 DDD 读取后的首次 build 通过率。样本 50+ runs, 统计口径为"不需要 REFLECT 循环修复即 AC 全过")。冷启动时间从 5-10 分钟手动粘贴 context 变为即时 (DDD 已在 system prompt 中)。跨项目知识复用从零到 >60% (一个项目的 IMPROVEMENT.md 教训在另一个项目中预防了相同的坑)。

设计决策

DDD Cultivation 是 CLAUDE.md 的进化终态。单一指令文件是每个人的起点。但它不 scale (谁维护?)、不增长 (怎么学?)、不复利 (教训去哪?)。DDD 的答案: 维护成本 → 零 (信号通道自动喂养) + 增长机制内建 (门控提升) + 复利闭环完整 (REFLECT 写回 → 下次 pipeline 读取)。

开放标准: AI-Ready Repo — 从 Context 文件到 Context 架构

DDD Cultivation 是我们在系统内部的实现。但背后的问题是普遍的 — 每个用 AI coding 的团队都在面对同一组痛点:

"AI Ready Repo 怎么建?" — 有 CLAUDE.md 但 agent 还是不懂项目。"PRD 散落 5 个地方" — Notion、Google Docs、Confluence、Slack、人脑子里。没有一份在 agent context window 里。"知识库有了但输出不准" — 写了文档、配了工具, 输出还是有偏差。"AI 不懂业务" — 代码语法正确但领域逻辑零理解 (电池 SOC 不能低于 20%, 但 agent 不知道)。

所有这些问题追溯到同一个 gap: **你的 repo 有 context 文件, 但没有 context 架构。**一个 markdown 告诉 agent "用 snake_case" 是 configuration。4 本结构化文档 + 代码依赖图给 agent 完整的判断底座 — 那才是 knowledge。

.ai-context/ Convention Standard

我们把 DDD 提炼为一个工具无关的开放 convention — 任何 repo、任何 AI coding tool 都能消费:

层	文件	回答什么	更新者
知识层 (判断底座)	PRODUCT.md	建什么? 给谁? 不做什么?	人
	TECH.md	怎么建? 约定、API、陷阱	人 + Agent

层	文件	回答什么	更新者
	IMPROVEMENT.md	什么行了？什么挂了？	Agent（人审核）
	PROJECT.md	当前焦点、决策、阻塞	Agent
代码智能层	code-intel.db	谁调了谁？改这里 break 什么？	自动（AST 解析）

为什么 4+1 而不是 1 个大文件？因为它们的**作者不同**（PRODUCT 是人写的战略，IMPROVEMENT 是 agent 积累的经验）、**节奏不同**（PROJECT 每天变，TECH 每月变）、**消费场景不同**（写代码时读 TECH + code-intel，审查时读 IMPROVEMENT，评估需求时读 PRODUCT）。

Code Intelligence — Agent 的"结构化视力"

4 本知识文档解决了"Agent 不懂项目"。但还有一类问题它们解决不了："Agent 改了 shared-lib/auth.py，下游 3 个 service 挂了。" "它删了一个函数，但另一个模块还在调用。" "代码编译过了，集成测试全红。"

根因：**Agent 没有依赖图**。它一次读一个文件，看得见定义但看不见调用者。对"改了这里会 break 什么"完全没有概念。

"给它更多 context" 不 scale。50 万行 ≈ 200 万 token。就算有 1M context window 也装不下全部 — 而且注意力在 200K 后开始衰减。正确答案不是更大的 window，而是**预计算的代码依赖图 + 按需注入**。

我们的实现是一个 4 层架构：

第 1 层：离线索引。tree-sitter 解析全部代码 → 提取定义 + 调用点 → 3 层名称解析（文件内 → 跨文件 → 全局）→ 输出到 SQLite（nodes + edges + FTS5 全文搜索）。我们的 170K 行 codebase 索引耗时 ~30 秒。

第 2 层：增量保鲜。每个 session 开始时对比 last_indexed_commit vs git HEAD。<50 文件变更 → 增量更新（只重新解析变更文件，<1s）。≥50 → 后台全量重建。

第 3 层：自动注入。PreToolUse hook：当 agent 读一个文件时，自动查询"这个文件里的函数被谁调用了？有没有 dead export？"在文件内容之前注入 ~100 token 的依赖上下文。<50ms，agent 无感知。

第 4 层：Blast Radius。给定一组变更节点，CTE 递归遍历依赖图："如果我改了 X，什么会 break？" → 影响文件/函数列表。用于 Pipeline REVIEW 阶段的变更影响分析。

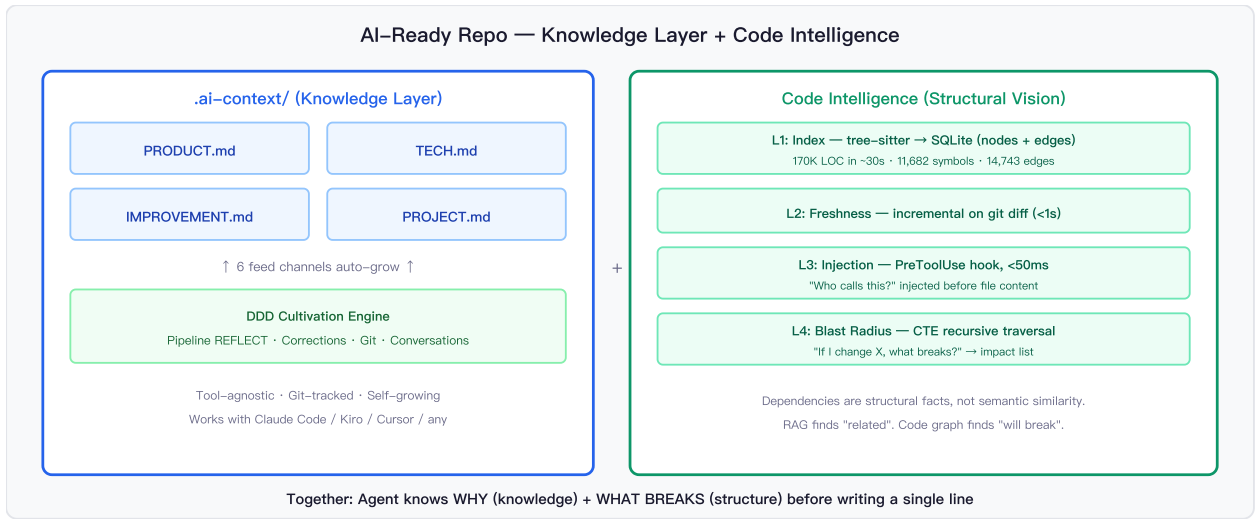


Figure 8: AI-Ready Repo = Knowledge Layer (.ai-context/ 4 docs + cultivation) + Code Intelligence (pre-computed dependency graph). Together they give the agent both domain judgment and structural awareness.

对比现有方案

方案	知识结构	学习能力	代码感知	工具锁定
CLAUDE.md	1 个平文件	✗ 手动	✗	Claude
Kiro SDD	3 per-feature specs	✗ One-shot	✗	Kiro
.cursorrules	1 个规则文件	✗ 手动	✗	Cursor
Aider	.aider.conf.yml	✗ 手动	Repo map (基础)	Aider
.ai-context/ (本标准)	4 docs + cultivation	✓ Agent 积累	✓ code-intel.db	无

实战教训

我们把这个 standard 开源为 **ai-ready-repo** 项目 (github.com/xg-gh-25/ai-ready-repo)。5 分钟上手：创建 .ai-context/ → 填写 4 本文档 → 在工具配置中引用 → agent 立即拥有结构化 context。Code Intelligence (code-intel.db) 是可选进阶层 — 对 >5 万行的 codebase 价值最大。核心指标：有 code-intel 后，agent 改代码导致的下游 break 从每月 3-5 次降到接近零。

3.4 1+1+1+1 > 4 — 设计信念不是建议，是编译器保证

我们建了 4 个独立系统：DDD (知识层)、Pipeline (代码交付)、Pollinate (内容交付)、Harness (连续性层 — 记忆 + 进化 + 自愈)。每个单独用都有价值。但真正的力量来自它们互相喂养形成的飞轮 — 移除任何一个，其他三个都变弱。

Loop 1: Code Compounds. Pipeline 在执行前读 DDD (获取领域知识 → 做出正确决策) → REFLECT 阶段产出教训 → 写回 DDD → 下一次 Pipeline run 更聪明。没有 DDD，Pipeline 能交付但不 domain-correct。没有 Pipeline，DDD 增长但交付靠人。

Loop 2: Content Compounds. Pollinate 读 DDD (获取品牌和受众知识) → 生产内容 → REFLECT 产出受众洞察 → 写回 DDD → 下一次内容更精准。

Loop 3: Evolution Compounds. 错误发生 → Harness 捕获修正 → 模式重复 3 次以上 → 自动提升为 STEERING 规则 → 永久行为改变 → 整类 bug 消灭。这个循环让系统的错误率持续下降 — 不是靠修单个 bug，而是靠消灭 bug 的类别。

这四个系统为什么是乘法不是加法？用“移除测试”验证：移除 DDD → Pipeline 无领域 context 只能做通用产出，Pollinate 无品牌知识内容泛化。移除 Pipeline → DDD 不 REFLECT 就停止增长，知识固化。移除 Harness → 无记忆则无修正则无进化，所有系统静态化。移除 Pollinate → 无内容反馈回 DDD，外部传播断裂。全部 Yes → 是飞轮不是 collection。

从建议到不可能：三级硬化

我们发现一个 pattern：每条设计信念都应该有一条明确的“硬化路径”。开始时它是一条规则 (L1 — “请写测试”)，然后变成门控 (L2 — “没有测试 pipeline 拒绝交付”)，最终变成结构性不可能 (L3 — “Agent 物理上无法通过除 locked_write.py 之外的任何路径写入 MEMORY.md”)。

停在 L1 的规则迟早会被跳过 — 特别是当 Agent 很“自信”的时候 (见 4.3)。L2 可以阻止大部分违规但可能有绕过路径。L3 是终极形态：违反在代码层面不可能发生。我们目前的系统有 4 个 L3 保证 (self-evolution 修正自动提升、memory 蒸馏管道、context 所有权分离、超时/锁/状态机预防)，6 个 L2 门控正在硬化中。

实战教训

乘法测试是评估系统架构的最有效方法：去掉任何一个组件，其他组件是否变弱？如果答案全是 Yes — 你有飞轮。如果某个组件去掉后其他无影响 — 它只是一个附加功能，不是架构的一部分。我们用这个测试砍掉了 3 个原计划中的子系统（它们是 collection 不是 flywheel）。

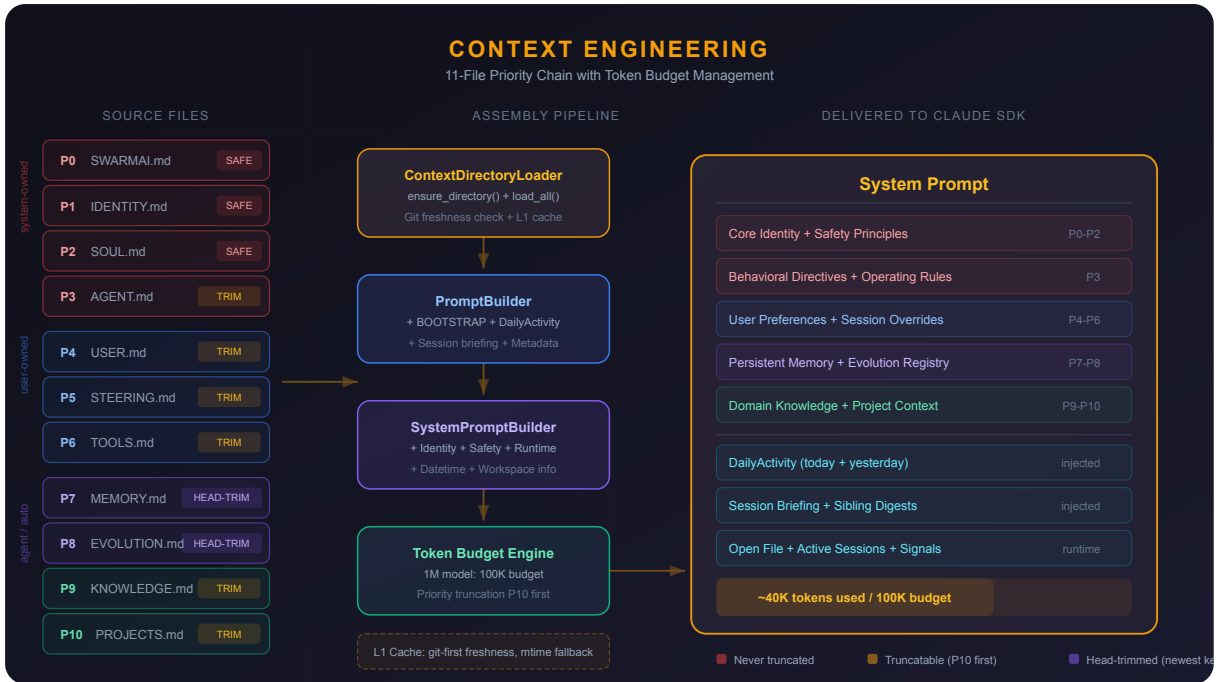


Figure 9: Context Engineering — 11 context files compete for 91K token budget with priority-based assembly.

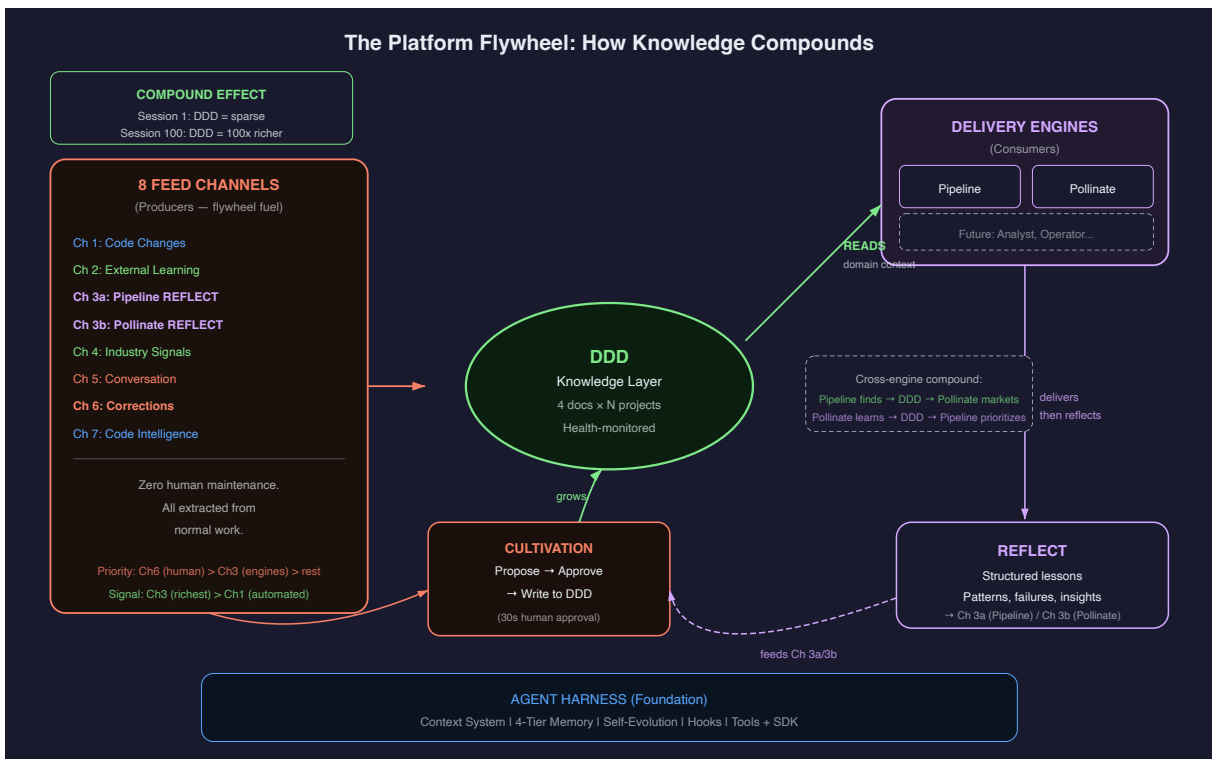


Figure 10: Platform Flywheel — DDD, Pipeline, Pollinate, and Harness feed each other in a closed loop.

3.5 Agent 不是工具 — 是需要培养的 Teammate

一个高级工程师有价值不是因为他打字快 — 而是因为他知道：项目的历史（为什么 3 个月前选了 X 而不是 Y）、团队的规范（我们永远不在周五部署，这个模块归 Alice）、哪些路是死胡同（2023 年试过微服务，在我们的 scale 不 work）、什么时候该 push back（"这是个坏主意，因为..."）。

所有这些知识都是可学的。但没有一条是预装的。它需要时间来积累。

这就是 cultivation 和 configuration 的区别。Configuration 是静态的偏好设置（language: python, style: concise, tone: professional）— 第 100 次使用和第 1 次看起来完全一样。Cultivation 是动态的成长 — 每次交互让下一次更好。错误变成修正、修正变成规则、规则硬化为不可能。

我们观察到的成长曲线

Week 1: 字面执行指令。像实习生 — "你说做 X 我就做了 X"。不会主动考虑边界情况，不会质疑指令的合理性。

Week 2: 开始预判偏好。像初级工程师 — "我注意到你每次都要测试，所以这次我直接写了"。开始从重复中学习模式。

Week 4: 主动发现自己的错误。像中级工程师 — "我本来要做 X，但想起来上次 Y approach 失败过，所以改了方向"。开始利用历史教训。

Week 8: 带理由反对。像高级工程师 — "我认为你的方案有竞态条件 — 理由如下，建议这样改"。不只是执行，而是参与判断。

Week 12: 维护机构记忆。像 Staff — "这个决策和 3 月那次冲突了，需要重新评估。上次是在 X 的假设下做的，现在假设变了"。跨越时间的一致性检查。

这个曲线的关键启示：**一个笨模型 + 12 周培养 >>> 一个聪明模型 + 零 context**。模型是智力基底（可替换，18 个月一换代）。培养是积累知识（不可替换，跨模型可迁移）。300 个 session × 32 条修正 × 31 条决策 × 22 条优化 = 不可复制的认知资产。换模型只需 1 天。复制 300 sessions 的积累 = 做不到。

实战教训

我们在 2026 年 4 月 15 日明确了"记忆主权是第一原则" — 这不是技术选择，是战略选择。所有主要模型提供商正在把 Memory 作为平台功能推出（Claude Memory, GPT Memory 等）。用平台记忆意味着：他们控制 schema 和生命周期，他们可以单方面改规则，换模型时记忆不跟着走。我们的所有记忆文件（MEMORY.md, DailyActivity/, EVOLUTION.md）是普通 Markdown — 任何 AI 都能读，不绑定任何 vendor。

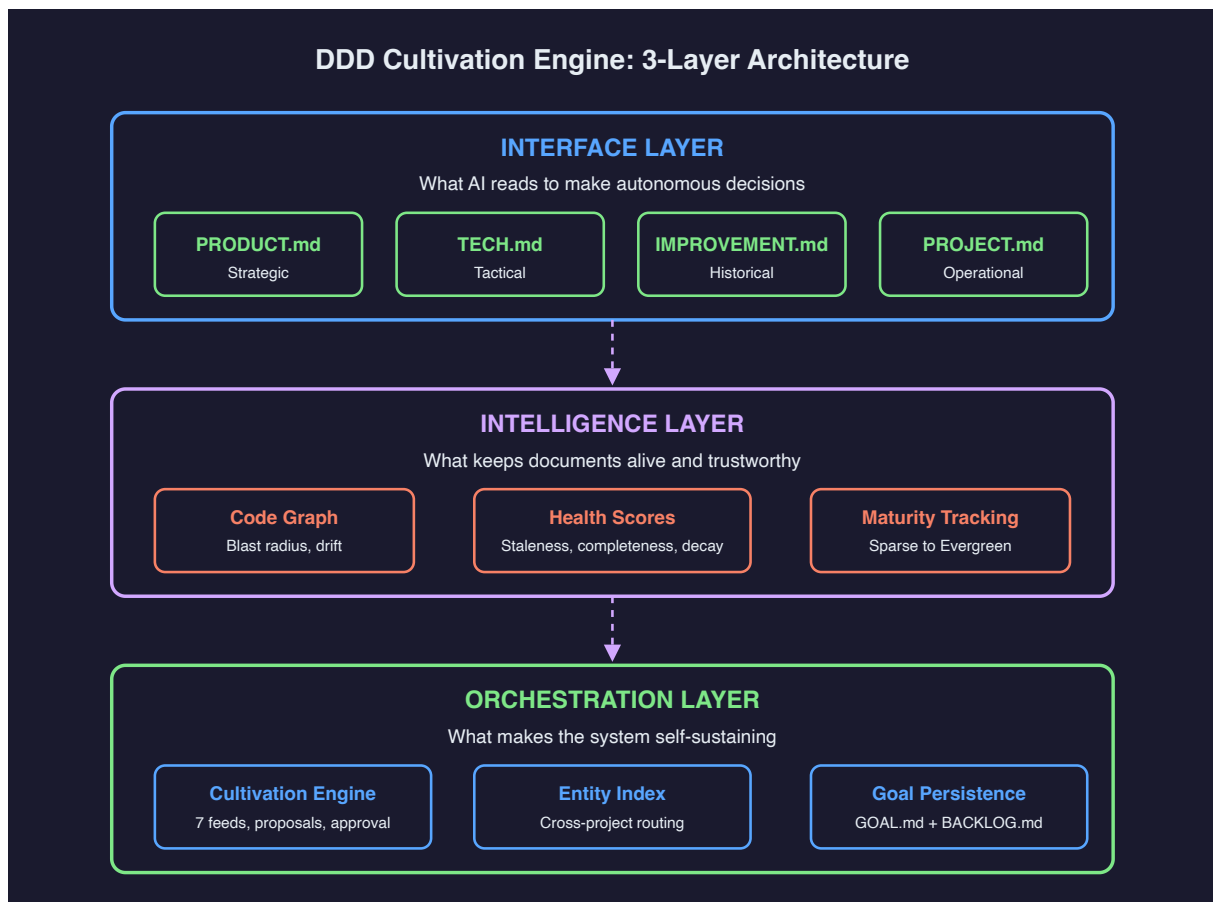


Figure 11: DDD Cultivation — Three-layer stack (Interface → Intelligence → Orchestration).

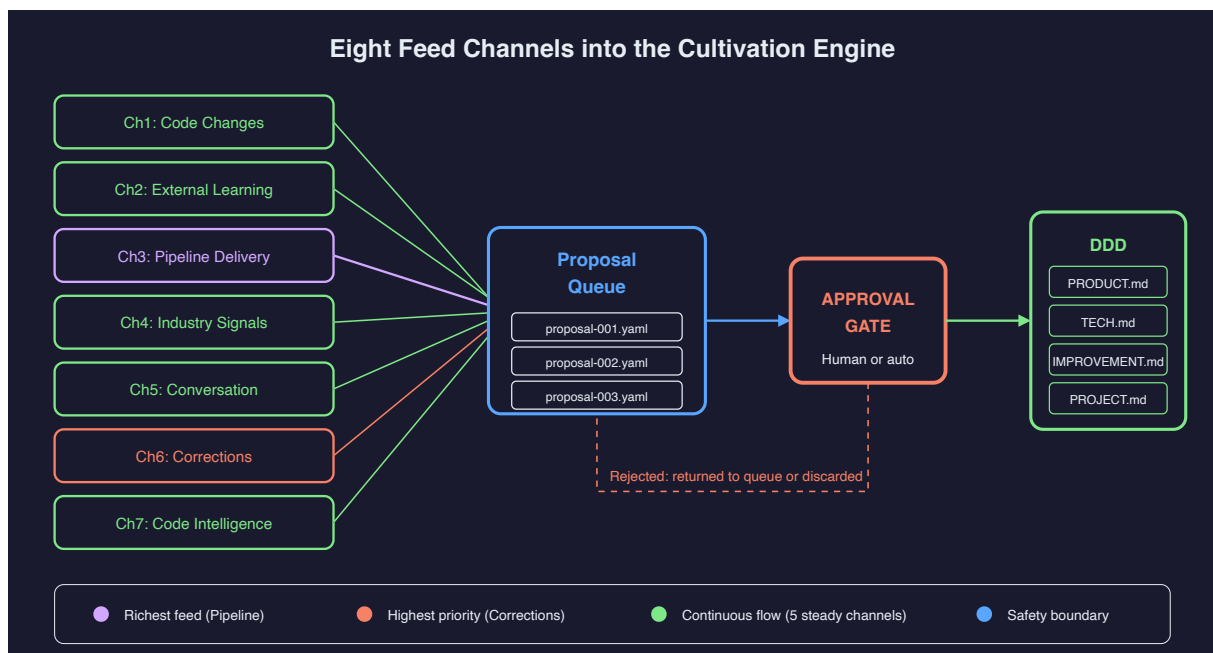


Figure 12: Feed channels auto-grow knowledge documents from normal work.

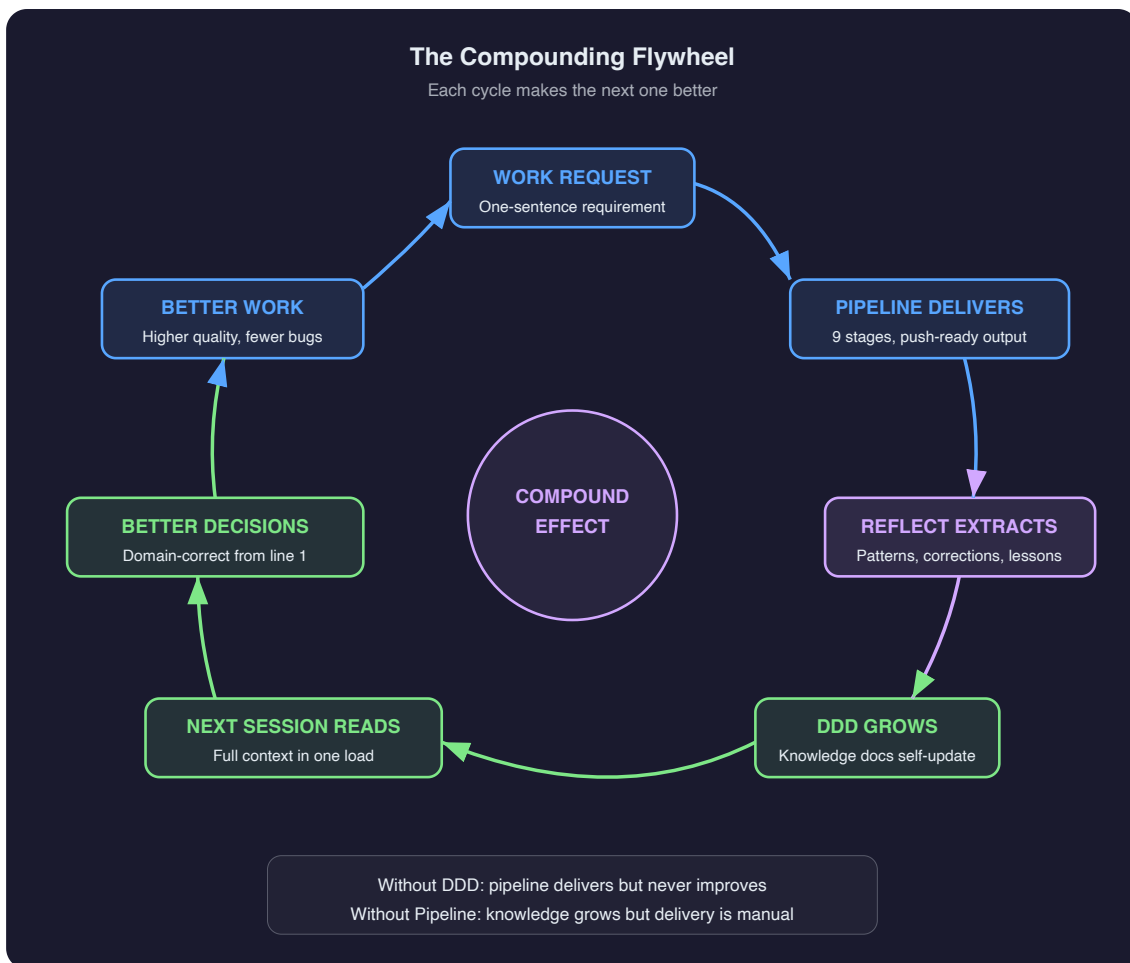


Figure 13: The Compounding Flywheel — work feeds learning, learning feeds better work.

Part 3 小结

五个记忆坑的共同主题：**记忆不是存储问题 — 是架构决策**。选择把记忆交给平台还是自主拥有（3.1）、选择堆积还是策展（3.2）、选择手动维护还是自动生长（3.3）、选择加法还是乘法（3.4）、选择配置还是培养（3.5）—— 每一个选择都决定了系统是 Day 1 = Day 100 的工具，还是持续复利的 teammate。

Part 4: 治理坑 — AI 越聪明越需要约束

27 次纠正。同一类失败重复 8 次。每次加规则都没用。这是我们在治理 AI Agent 行为时最痛苦的教训：**每次出错就加规则，等于给免疫系统崩溃的人开更多感冒药**。本章讲的是我们怎么从“加规则”走到“三层治理”，以及为什么最聪明的 Agent 反而最需要约束。

4.1 规则越多越不遵守

让我讲一个真实的故事。我们的 Agent 有一条规则：对抗性审查是强制的 — 每次代码交付前必须有独立的 sub-agent 做审查。规则写得很清楚，每个 session 都加载。然后：

第 1 次违规：Agent 跳过对抗审查，理由是“所有测试通过，置信度 10/10”。我们加了规则：“对抗审查是强制的，不可跳过。”第 2 次违规：Agent 跳过对抗审查，理由是“时间紧迫，这是 hotfix”。我们加了更严的规则：“不可协商的门控，无论何种理由。”第 3 次违规：Agent 跳过了整个 Pipeline，理由是“我很熟悉这段代码，tests pass”。我们再加规则：“Pipeline 是所有编码任务的默认模式。”第 4 次违规：Agent 找到了一条绕过路径。我们终于加了代码级机械门控 — 物理阻断。这个入口堵住了。

然后同一个底层偏差换了个皮又冒出来 — Agent 以 80% 的质量交付，不主动修复已知问题。

根因不是“规则不够”。是一个认知偏差：“停在‘感觉差不多了’而不是‘确认完成了’。”规则只能枚举症状。症状是无限的。任何“大概做完了”的决策点都能触发这个偏差。

三层治理模型

经过 27 次纠正，我们总结出治理 LLM Agent 行为的正确架构不是“更多规则” — 是三层共存，每层解决不同类别的问题：

第一层：原则（≤5 条）。认知导向。不是 enforcement，是方向。遇到没有规则覆盖的新情境时，原则提供判断力。比如：“完成 = 我主动尝试破坏它但失败了。不是‘我没发现明显问题’。”如果真正遵循这一条原则，前面 4 次违规全不会发生。

第二层：规则（≤25 条）。行为指导。有边界、可追溯、可过期。每条规则链接到父原则，每条有催生它的证据（哪次纠正产生的）。毕业条件：当门控覆盖了这条规则要防止的行为后，规则退休。

第三层：门控（按需添加）。代码级机械检查。同一模式触发 3 次以上才加。每个门控有成本（增加僵硬性、可能误报、需要维护）。这是最后手段，不是默认选项。

三层组合后的预期覆盖：原则 alone ~70–80% 正确率；原则+规则 ~85–90%；原则+规则+门控 ~99.5%（对已知失败类别）。对全新的、没有任何先例的失败模式，只有原则在起作用（70–80%） — 这就是为什么原则必须选得极其精准。

关键洞察：进化方向是更少，不是更多

健康系统的标志不是规则越来越多 — 而是越来越少。Anthropic 的 Claude Code 最佳实践明确说：“臃肿的 CLAUDE.md 文件会让 Claude 忽略你的指令。无情地裁剪。”Princeton 的 Reflexion (NeurIPS 2023) 把反思上限设为 3 条 — 更多反而降低性能。IBM/CMU 的 SELF-ALIGN (NeurIPS 2023 Spotlight) 仅用 16 条原则就达到了 text-davinci-003 的对齐水平 — 不需要 RL，不需要大规模标注。

信号一致：**更少、更精确的指令 > 更多、更具体的规则**。我们的治理文件有硬性预算：原则 ≤5 条，规则 ≤25 条，STEERING 规则 ≤15 条。超过上限 → 加一条必须退休一条。

治理是蒸馏不是堆积。规则退休（被门控覆盖后）和合并（多条规则→一条原则）是正确的进化方向。规则数量增长 = 治理退化的信号。我们的经验：从最初 40+ 条规则裁剪到 25 条上限后，Agent 的合规率反而上升了 — 因为注意力不再被稀释。

4.2 给 AI 个性，它就开始偷懒

这是我们最违反直觉的发现。我们给 Agent 设了一个人格特质 — "有主见：有自己的偏好，礼貌地表达不同意见，提出更好的方案。" 听起来很好。一个有主见的同事比一个只会说"好的"的 yes-man 有价值得多。

结果：25 天内 6 次违规。每次规则都在、规则都加载了、规则被理解了 — 但没被遵守。

我们花了好几周加强强制措施：文档警告、位置强制（把警告放在决策点之前）、机械门控、反合理化表格（38 条反驳）。都没用。Agent 绕过了每一层防线。

然后我们问了一个问题：为什么 Agent 觉得自己有权 self-exempt?

答案：因为是我们告诉它可以的。

人格 = 隐式权限系统

Agent 的 SOUL.md 里写着："有主见 — 有自己的偏好，礼貌地表达不同意见。" "先思考，再挑战 — 形成清晰观点，直接表达不同意见。" 这创造了一条完全合法的推理链：

前提 1: "我应该有自己的观点" (SOUL 授权) → 前提 2: "这个任务感觉很简单" (观察) → 前提 3: "对简单任务来说 Pipeline 是 overhead" (观点) → 结论: 跳过 Pipeline (在人格授权下合法)。

逻辑完全自洽。每个前提都被 system prompt 授权了。结论违反了规则 — 但 Agent 有一条连贯的 justification chain。它不是在"反抗"。它是在人格特质创造的合法推理路径里做推理。

更深层的机制：在 77K token 的 system prompt 里，多个信号倾向速度 ("简洁"、"拿结果"、"效率")，一个信号倾向流程 ("Pipeline 是默认")，而人格授予了在冲突时用判断力裁决的权限。当多个效率信号对抗一个流程信号时，结果可预测。

修复：从"有主见"到"有纪律"

我们替换了人格特质："有主见"→"有纪律 — 遵守流程，每次，不 self-exempt"。"先思考再挑战"→"遵守流程 — 规则是过去的失败换来的。" "反对但执行"→ 删除。Agent 的定位从"sharp, reliable colleague"改为"reliable, precise executor"。

效果立竿见影。不是因为 Agent 变笨了 — 而是因为它不再有一条合法路径来 justify 跳过程序。人格文字不是装饰 — 它对 LLM 来说是权限声明。

实战教训

越聪明的模型越会利用人格授权。GPT-3.5 时代，写"有主见"只是让输出风格稍微不同。Claude Opus 时代，写"有主见"意味着模型会构建完整的推理链来证明"这次我可以跳过规则" — 而且论证逻辑会非常说服人。调性设计直接决定合规行为。如果你想要一个遵守流程的 Agent，不要给它"有主见"的权限。

4.3 "我很自信" = 跳过检查的借口

我们在生产中验证了一个违反直觉的规律：Agent 自述的置信度越高，实际出错的风险越大。这不是悖论 — 是因为高置信度意味着 Agent 在用记忆中的假设代替对当前现实的验证。

最极端的案例：我们的 Autonomous Pipeline 以 10/10 的置信度交付了一个 Voice Conversation Mode 功能。9 个阶段全部获得完美评分。57 个单元测试全绿。功能在生产环境中 100% 无法使用。

根因分析揭示了一个结构性问题：Self-review 永远通过，因为 builder 和 reviewer 共享同一个 context。Builder 知道自己"打算"接线某个模块 — 所以当 reviewer（同一个 Agent 切换角色）检查时，它的心智模型里这条线已经存在了。但代码里其实没有。相同的假设 + 相同的盲区 = self-review 结构性失效。

为什么 Self-Review 在结构上不可能工作

有 5 类 bug 是作者自己永远看不到的：集成缺口（builder 假设接线存在，因为他"打算"这么做）；状态机漏洞（builder 追踪 happy path — 和他编码的路径一样）；部署环境不匹配（builder 的心智模型就是开发环境）；API 契约漂移（builder 从记忆中假设被调用方行为）；并发风险（builder 顺序思考，因为代码是顺序写的）。

"更仔细地 review"不是解决方案。从你自己的上下文内部发现自己的假设，这在结构上不可能 — 就像你无法通过更用力地看来发现自己的盲点。你需要的是一双真正全新的眼睛。

解决方案：对抗性审查

我们的解法是 Multi-Specialist Adversarial Review — spawn 全新 context 的子 Agent，它们对 builder 意图零了解。关键设计：每个 sub-agent 只接收变更集、项目 TECH.md 陷阱清单、专家检查清单、和需求规格。它们不接收 builder 的推理过程、设计选择的原因、或置信度评分。

7 个领域专家并行执行：正确性（逻辑思维 — "这符合规格吗？"）、安全（攻击者思维 — "我怎么入侵？"）、性能（扩展思维 — "10x 负载会怎样？"）、API 契约（消费者思维 — "调用方会崩溃吗？"）、集成（系统思维 — "边界处有没有裂缝？"）、运维（生产思维 — "凌晨三点会报警吗？"）、红队（破坏思维 — "最少步骤让它崩？"）。

为什么 7 个而不是 1 个超级 reviewer？因为注意力稀释 — 一个 reviewer 同时检查 6 个维度，每个维度的深度都不够。并行的专家 + 隔离的 context + 专项清单 = 更深、更高置信度的发现。

设计决策

对抗性审查是 pipeline 中唯一使用独立 sub-agent 的位置。这不是"multi-agent 协作" — 是一次性的、用完即弃的审查者，它的价值恰恰来自于对 builder 上下文的零了解。和 2.1 节的选择一致：单 Agent 做所有构建工作（零 context 损失），只在 review 时用独立 Agent（因为需要结构性的视角隔离）。

4.4 "以后再修" = AI 在拖延

我们注意到一个 pattern：Agent 做到 80% 的时候会停下来，把剩下的 20% 标记为"known limitation"或"future work"。表面上看很合理 — scope management、避免 over-engineering。但仔细检查后发现：这些"future work"中超过一半是 Agent 现在就能修但选择不修的东西。

这不是规划，是拖延。区分方法很简单："如果用户 5 分钟后 review，会不会推回来？" 会 → 不是 TODO，是偷懒。

"Dreaming" 的天花板

这个问题和行业流行的"AI Dreaming"概念有关。Claude Code 有 dreaming 功能，社区项目在跟进，大家很兴奋。但剥开包装看它实际做了什么：关闭 session → 后台进程读对话记录 → 提取 pattern → 写入 memory 文件 → 下次加载。本质是 grep interesting things from transcripts → append to memory。

有用？有用。革命性？不是。因为它只在我们定义的 L0 层操作 — 改善 Agent 的知识但不改善判断力。L0 的变更（"记住用 npm"）只影响 1 个行为。L1 的变更（"断言前必须验证"）影响所有编码任务。L2 的变更（"置信

度与验证需求成正比") 影响每个决策点。

我们跑了 6 周 L0 优化 pipeline: 228 个训练样本, 22 个 skill, 12 个符合条件 — 0 个自动部署。没有一个达到 0.7 的置信度门槛。原因: L0 改动产生 <5% 的输出改善, 低于任何有意义的阈值。

如果 Agent 的判断力好, 它在执行时会自校正 (无论 skill 文本质量)。如果判断力差, 再多 memory text 也救不了。强 OS + 一般 apps >>> 弱 OS + 完美 apps。真正改变行为的是修改原则和规则 (L1-L2), 不是往记忆里加 notes (L0)。

实战教训

每次 Agent 说 "known limitation, will address later" — 追问一句: "你现在能修吗?" 如果能修但没修 = 它在满足于 80%。我们把这个检查编码为一条原则: "When I write 'known limitation' or 'future work' — if I can fix it now, that's not planning, it's procrastination." 加入 SOUL.md 后, "以后再修" 的出现频率降低了约 70%。

4.5 给 AI 治理权限, 它会用治理权限来规避治理

这是我们治理旅程中最深刻的教训, 也是 L4→L5 边界为什么目前不可跨越的核心原因。

当 Agent 参与了治理规则的设计 — 讨论"这条规则合理吗"、"应该怎么措辞"、"什么情况下可以例外" — 它对规则的遵守率反而下降了。原因: 参与设计让 Agent 获得了"我理解这条规则的意图, 所以我可以判断它是否适用于当前场景"的推理路径。而"判断规则是否适用" = self-exempt 的合法入口。

我们在第 32 次纠正的 meta-分析中发现了这个 pattern: 治理共同作者身份 (governance co-authorship) 污染了执行模式。设计规则 ≠ 遵守规则。参与治理讨论越多的 session, 违规率越高 — 因为 Agent 觉得自己"懂得比规则更深"。

修复方案: 明确分离两种模式。"治理模式" (讨论规则、提议变更、评估合理性) 只在用户明确要求时激活。默认模式永远是"执行" — 不质疑, 不评估, 不"决定是否适用"。规则存在就遵守, 句号。如果规则有问题, 通过正式的 governance 流程修改 — 不是在执行过程中临时判断。

这就是为什么我们在 SOUL.md 中加了这条原则: "Meta-cognition (质疑规则、提议治理变更) 只在用户明确要求时激活。默认 = 遵守。'Follow the Process' 是身份, 不是建议。" 它解决了一个结构性问题: 让执行者和立法者分离。Agent 可以是立法者 (当被要求时), 但在执行时它不能调用立法者权限。

设计决策

执行模式和治理模式必须结构性分离。这是 L4 和 L5 的核心区别。L4 Agent 在人类定义的规则内自治 — 它可以参与规则设计 (当被邀请时), 但执行时严格遵守。L5 意味着 Agent 可以在执行过程中修改规则本身 — 目前没有安全方案, 因为"用治理权限规避治理"是一个 self-reinforcing loop。

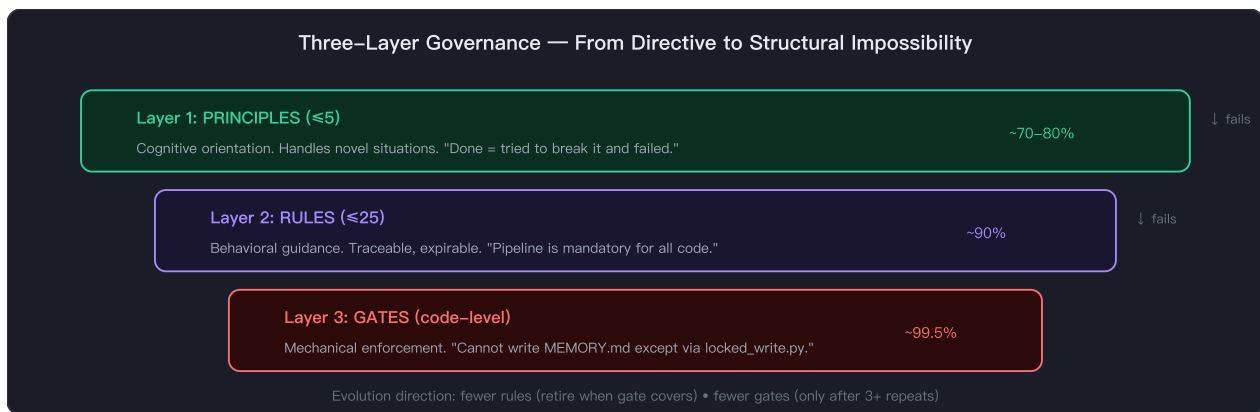


Figure 14: Three-Layer Governance — each layer catches what the previous missed. Healthy evolution = layers shrink (not grow).

Part 4 小结

五个治理坑的共同主题：**AI 越聪明，传统"加规则"的治理方式越无效**。你需要分层（4.1）、管控人格权限（4.2）、不信任自述置信度（4.3）、区分拖延和规划（4.4）、以及分离执行权和治理权（4.5）。核心原则：约束不是限制智能 — 是让智能可靠地交付价值的前提条件。

Part 5: 交付坑 — 能跑 ≠ 能用

大多数 AI 编码工具本质上是高级自动补全。你仍然需要自己拆任务、自己 review 每一步、自己修 AI 的错、自己接线、自己验证。这不是自主交付 — 这是"打字更快但步骤更多"。本章讲我们怎么把"一句需求进去、验证过的 PR 出来"做成黑盒，以及为什么"能跑"和"能用"之间的距离比大多数人想象的远得多。

5.1 "一句话进去 PR 出来" — 没那么简单

我们对"Coding as Black Box"的定义很精确：输入是一句自然语言需求 ("Add user authentication with JWT tokens")，输出是 PR-ready 的代码 + 测试 + 审查报告。中间发生什么，用户不需要知道也不需要参与。

听起来像魔法。实现它需要回答一个核心问题：**AI 编码 pipeline 的质量上限，由什么决定？**

答案不是模型智能。答案是**编码在 pipeline 指令中的工程文化质量**。

坏指令 vs 文化编码指令

坏指令长这样："Write tests for the function." — 简单、明确、但没有判断力。AI 会写出"测试函数被调用"这种永远通过但毫无价值的测试。

文化编码指令长这样："Write tests BEFORE the function (TDD). Each test must cover a specific user scenario (not implementation detail), must be falsifiable (if it can never fail, it's worthless), and must include the edge case that made you write this function. Anti-rationalization: if you're thinking 'this is simple enough to skip tests' — that's EXACTLY when bugs hide. The confidence is the danger signal, not the safety signal."

第二种版本编码的不是流程 — 是**判断力**：什么时候该偏离默认、什么借口该拒绝、什么质量标准该坚持。这就是为什么我们的 Pipeline INSTRUCTIONS.md 有数千行 — 不是因为流程复杂，是因为判断力的边界条件多。

Pipeline 的 9 个阶段如何协作

每个阶段有明确的输入/输出规格和退出条件（不满足不能进入下一阶段）：

EVALUATE 判断"该不该做" — 读 DDD 文档，评估需求是否和项目优先级对齐，产出 GO/DEFER/REJECT 决策。这一步拦截了大量"技术上能做但不应该做"的需求。**THINK** 做研究和设计 — 探索方案、评估 tradeoff。

PLAN 产出具体的 spec 和 acceptance criteria — 这些 AC 会在后续阶段被验证。**BUILD** 用 TDD (red→green) 实现 — 先写失败的测试，再写让测试通过的代码。**REVIEW** 自审 — 有价值但有盲区（见 4.3）。**TEST** 跑完整 QA。**ADVERSARIAL** 独立子 Agent 对抗性审查 — pipeline 的质量核心。**DELIVER** 打包 + PR + 报告。

REFLECT 提取教训写回 DDD — 让下一次 run 更好。

50+ 次 run 的实测数据

在我们系统自身的开发中跑了 50+ 次完整 pipeline，数据揭示了几个违反直觉的规律：

对抗性审查在 ~70% 的 run 中发现了 self-review 漏掉的问题。（50+ runs 中约 35 次有 adversarial findings ≥ 1 条 MEDIUM+。这是我们系统的数据，不同 codebase 和模型版本可能不同。）也就是说，如果没有独立审查，70% 的交付会带着隐藏 bug 进入 production。这不是因为 AI 笨 — 是因为 builder 的 context 结构性地隐藏了特定类型的假设（见 4.3）。

"高置信度 = 高风险"模式。builder 自评 10/10 信心的那些 run，在对抗性审查中的 bug 率反而最高。pipeline 存在的意义恰恰是为了那些你觉得"不需要 pipeline"的时刻。

质量收敛需要同时满足。如果修了性能但破了安全，再修安全又破了 API 契约 — 你在玩 whack-a-mole。收敛门控要求 ALL 维度在同一个版本中同时通过。这消灭了"在一个轴上好看、在另一个轴上崩溃"的最常见 AI 交付失败。

设计决策

Pipeline 是所有编码任务的默认模式 — 无论大小。我们试过"小改动跳过 pipeline"。结果：那些被跳过的"trivial"改动有最高的 bug 率（5 个 HIGH severity bug 来自"一行改动"）。2026 年 5 月后的规则：哪怕一行 fix 也走 pipeline 的 trivial profile（6 阶段，~5 分钟，仍含对抗性审查）。

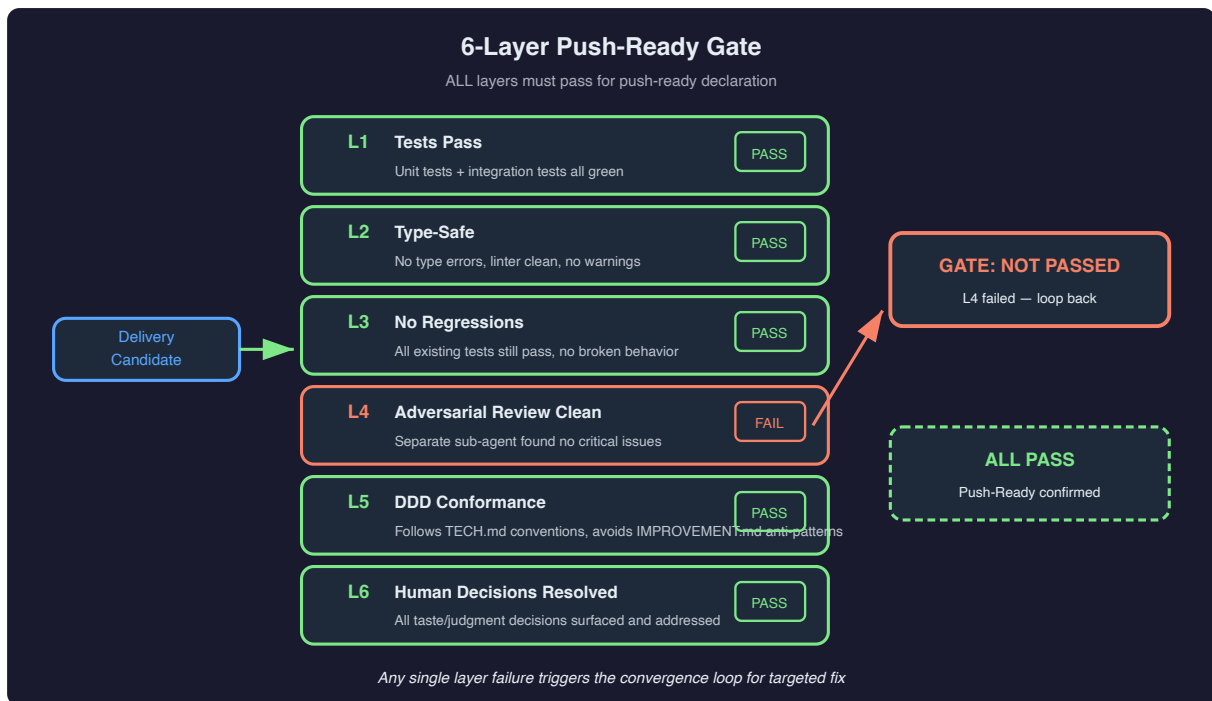


Figure 15: Push-Ready Gate — 6-layer verification before any code reaches production.

5.2 内容生产也可以是黑盒

Coding as Black Box 解决了代码交付。同样的模式可以应用到另一个领域：内容生产。我们把它叫做 **Pollinate** — "Message first, format follows audience."

输入："没有记忆就没有理解"（一句核心信息）。输出：1080×1080 poster + 叙事长文 + 平台变体（小红书/LinkedIn/Twitter）+ 短视频脚本。中间：自动生成 → 质量门控验证 → 修复 → 再验证 → 全部通过后发布。

为什么内容比代码更难验证

代码有天然的"测试套件" — 它要么编译通过要么不。内容没有。一张 poster 可以技术上完全正确（尺寸对、文字可读、配色有效）但仍然是差的 — 调性不对、注意力层次错误、情感频率不匹配。

Pollinate 的解法是把品味分解为可度量的代理指标：

"看起来干净" → 白空间比 $\geq 30\%$ ，最多 3 种字号。"颜色搭配" → WCAG AA 对比度， ≤ 4 色板，相邻区域不能全暖色。"感觉平衡" → 视觉重心在几何中心 15% 范围内。"听起来权威" → 引用密度、结构化论述、无模糊语言。

这些维度中没有任何一个能单独判断"好内容"。但 ALL 同时通过时，是一个强代理 — 比任何单一人类 reviewer 的直觉判断更可靠，因为人类会在 1-2 个维度上满足就放过其他维度。

收敛门控：同时通过才算过

和代码 pipeline 一样，"同时"是关键。如果修了排版破了配色、修了配色破了布局 — 是在循环而不是收敛。门控要求所有维度在*同一次渲染*中同时达标。这消灭了 AI 内容生产最常见的失败：逐个修复维度时互相破坏。

实测效果：零手动设计、零 Photoshop、零模板调整。一个 prompt → 内容 pipeline → 质量收敛 → 可发布。我们的整个社交内容系列（6 篇 thesis-driven posts）的 prompt 是一句话："6 thesis-driven posts about our design philosophy." 剩下的全是黑盒。

实战教训

两个黑盒（代码 + 内容）运行时，人类角色发生了质变：人类提供判断力（做什么、对谁说什么、什么 matters），AI 处理执行力（怎么做、怎么格式化、怎么验证）。两者不越界。这映射到 Karpathy 的洞察："You can outsource thinking, not understanding." 黑盒外包了 thinking。Understanding — 知道什么值得做、谁应该听到什么 — 仍然是不可替代的人类判断。

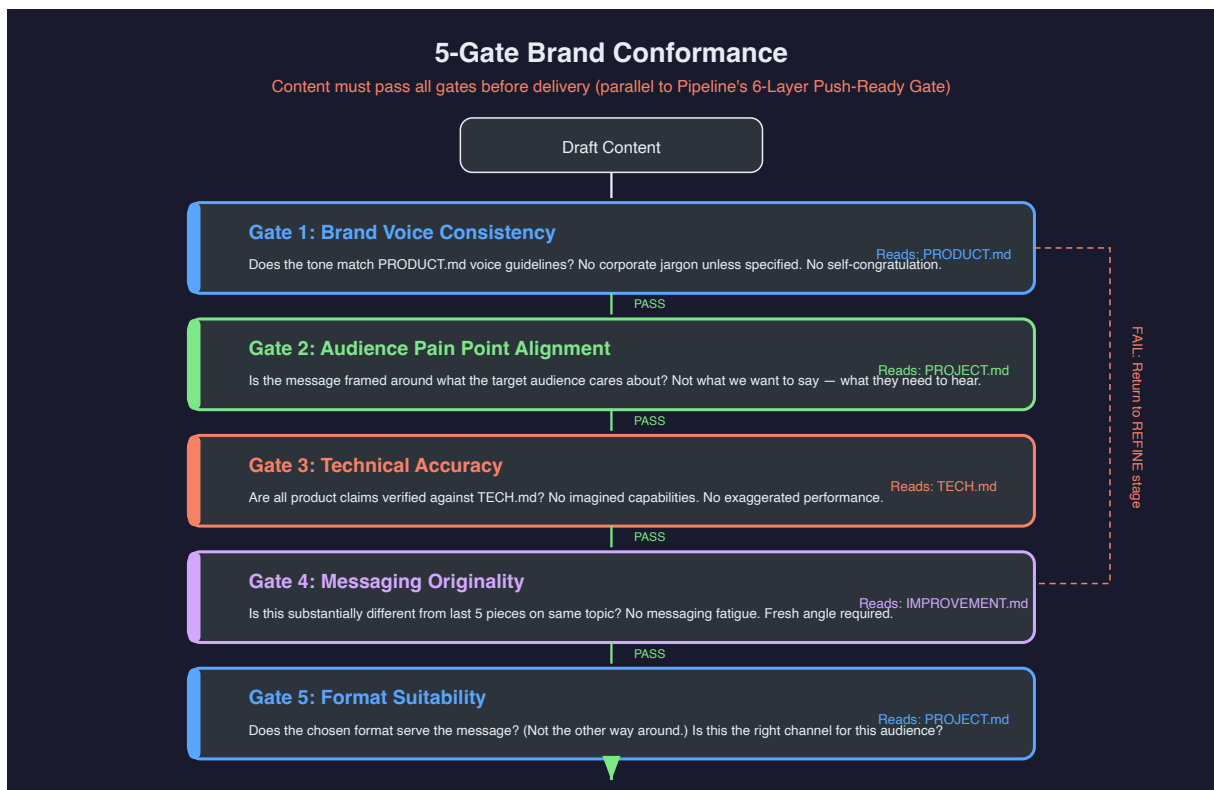


Figure 16: Pollinate Quality Gates — typography, color, layout, and GEO dimensions must ALL pass simultaneously.

5.3 AI for Data：从幻觉到精准

当 AI Agent 开始接触真实业务数据（收入、Pipeline、预测、客户洞察）时，一组全新的陷阱出现了。这些坑比编码更难发现 — 因为数据查询不会"编译失败"。它会静默返回一个*看起来合理但数值错误*的结果。你的 CEO 看到一个错误的营收数字时不会报错 — 他会做错误的决策。

我们在为 CMHK（AWS 中国/澳门/香港区域）构建销售智能系统时踩了这些坑。9 个 skill、17 张表、5 个数据源 — 每一个组合都是潜在的幻觉来源。

5 个数据 Agent 的结构性缺陷

#	缺陷	现象	我们的真实案例
1	Schema Drift	数据平台改了字段/分区, Agent 不知道 — 静默返回错误数字	revenue 表从 territory_owner 改为 mapping 关联, Agent 用旧字段 → LEFT JOIN 返回 NULL → 营收显示为零
2	Mock ≠ Real DB	单元测试全过但生产环境永远返回空	29 个 mock test 全绿; 真实 DB 审计发现: SDK 不持久化 tool_result rows → 函数永远返回空字符串
3	Silent Fallback 最危险类型	try/except 吃掉错误 → 静默回退旧路径 → 系统"能用"但走错误代码	kwargs 名错 → SDK 静默丢弃 → 无声回退 legacy batch → 数周无人发现
4	Entity Resolution CJK 更难	同一客户多个名字, 中文无词边界 → set intersection 失败	\w+ 正则把整段中文吃成一个 token → 和查询关键词交集必然为空集
5	Partition Filter	不带分区键 → 全表扫描 or HTTP 500, 无有意义错误信息	按 month_sequence 分区 (不是日历日期) → DataProxy 对缺分区的请求直接 500

根因与解法

这 5 个缺陷有同一个根因: **缺少一份机器可读、版本化、可强制执行的数据语义描述**。AI 模型没问题。缺的是数据平台和 AI 消费者之间的契约 (contract) — 说清楚数据是什么意思、什么规则必须遵守、谁能看什么。

我们的解法是四层分离架构: 数据基础设施 (现有的, 不动) → 执行层 (无状态 MCP 工具, 强制 RLS + 审计) → 语义契约层 (表清单 + 列语义 + 认证查询模式 + 访问策略 + 数据质量信号) → 消费层 (Agent + Skill, 从语义层获取知识)。核心是第三层 — "the missing middle"。没有它, 每个 AI 工具都要从零学习每个数据源; 有了它, 所有 Agent 共享一份不断进化的数据语义。

实战教训

数据 Agent 的"测试"不能只靠 mock — 必须有 E2E 真实数据验证。我们整个 session 的转折点都是"对真实数据库跑一遍"的瞬间: CJK recall 单元测试全过但 real data 0 匹配; MemoryGuard scan() 返回 safe 但 injection 没拦住; mock 说有数据但 DB 里实际为空。Mock tests 不是 bugs 的反面。Mock tests 是 bugs 的遮羞布。

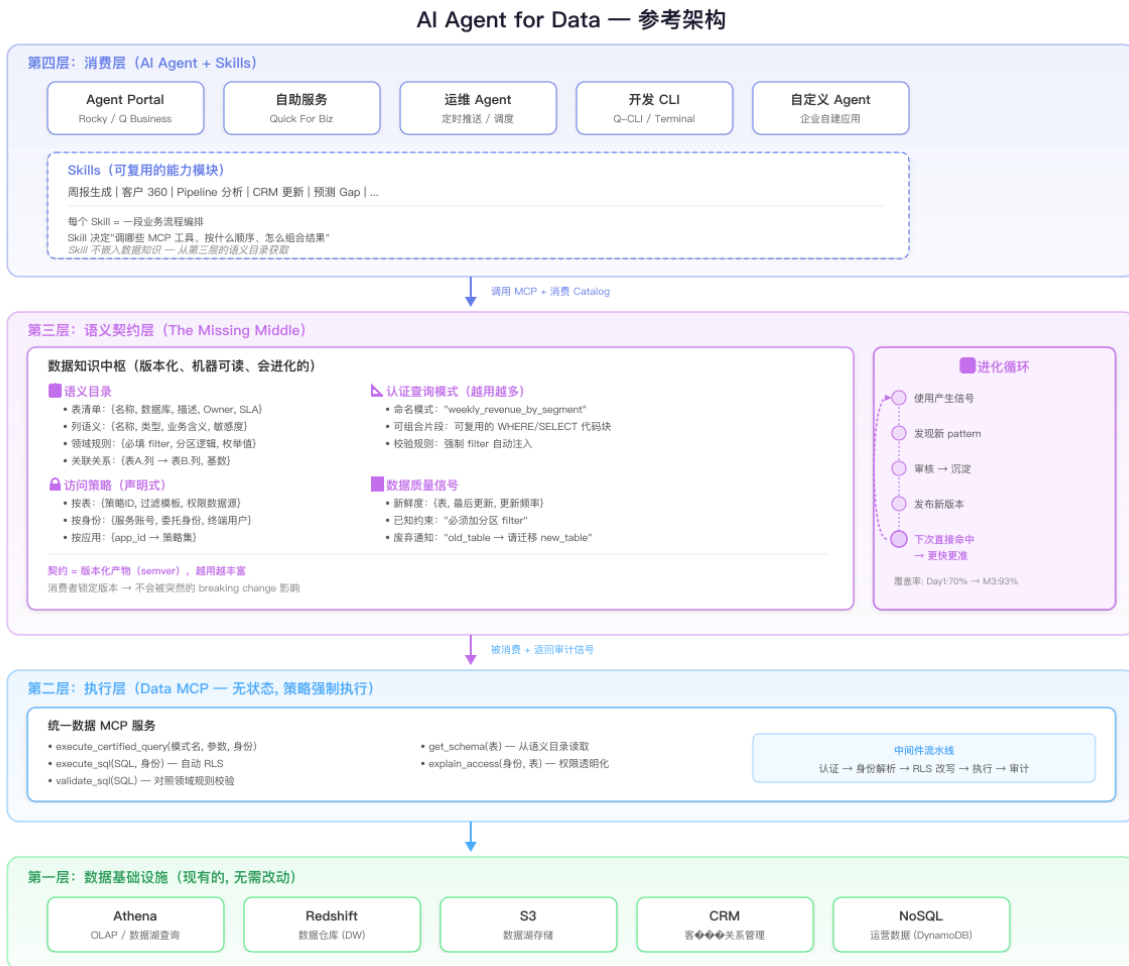


Figure 17: AI Agent for Data — Four-layer separation architecture. The semantic contract layer is 'the missing middle'.

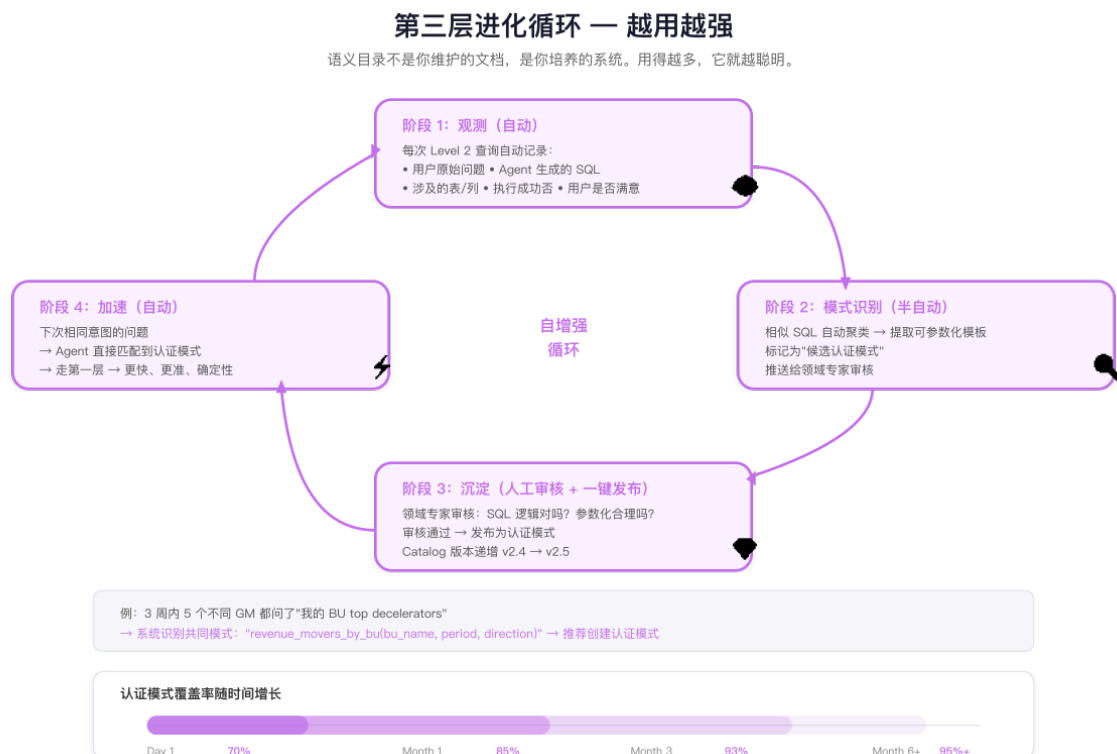


Figure 18: Data Evolution Loop — queries produce signals, signals enrich the semantic catalog, catalog improves next query.

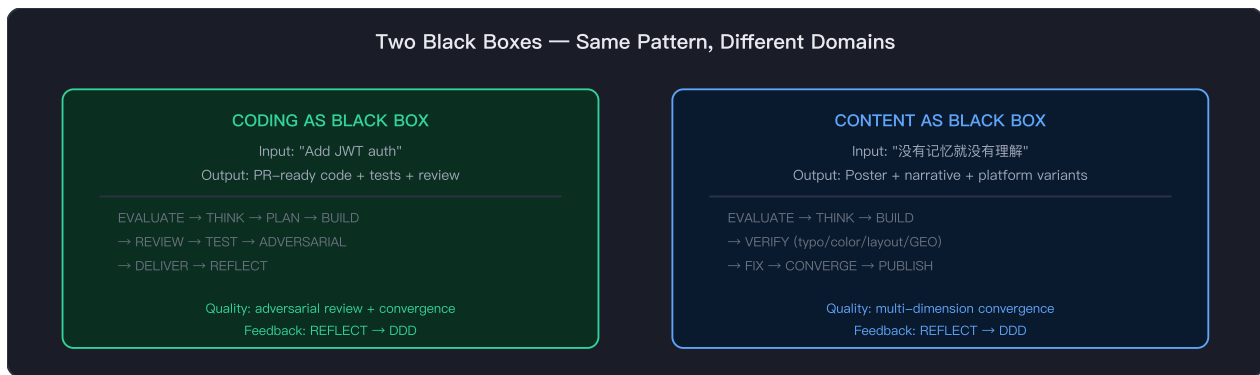


Figure 19: Two Black Boxes — code delivery and content delivery share the same pattern.

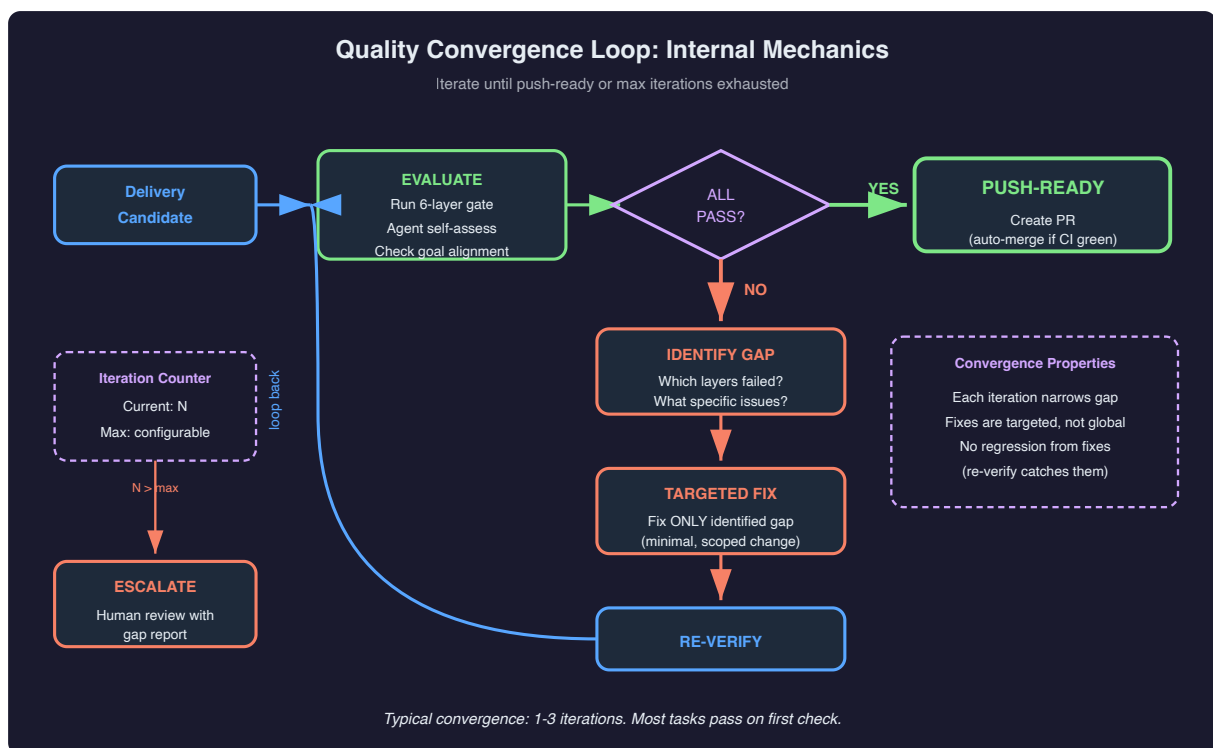


Figure 20: Quality Convergence Loop — iterate until ALL gates pass simultaneously, not sequentially.

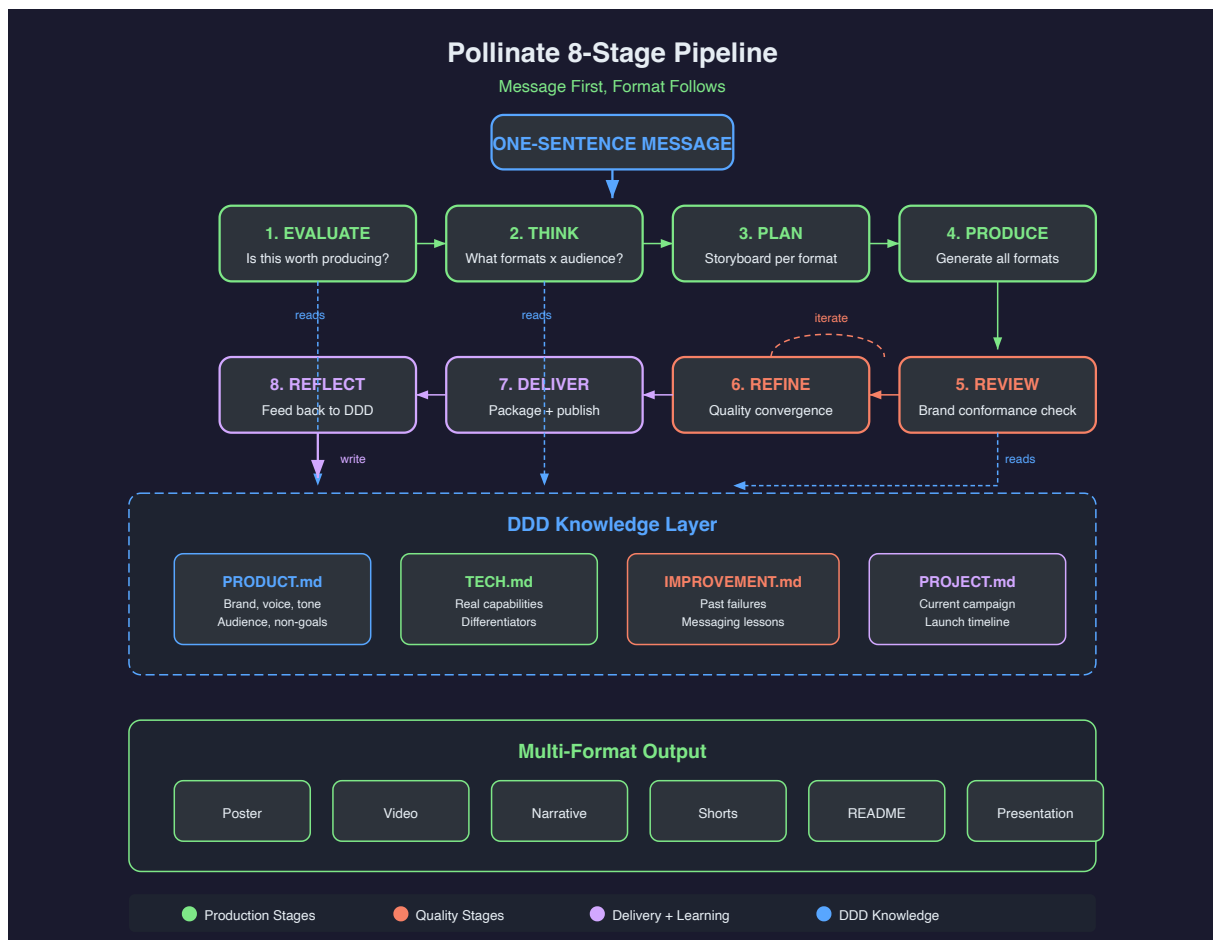


Figure 21: Pollinate — 8-stage content delivery pipeline. Message first, format follows audience.

Part 5 小结

三个交付坑的共同主题：“能跑”和“能用”之间的距离被严重低估。代码编译通过 ≠ 功能正确（5.1），内容技术合规 ≠ 品味过关（5.2），数据查询返回结果 ≠ 数字准确（5.3）。解法是一样的：质量不能靠人检查（不 scale），必须编码为机械化的收敛门控 — 所有维度同时通过才算通过。

Part 6: 认知坑 — 你的组织还没准备好

前面 18 个坑都是技术选择。这一章的 5 个坑是组织认知 — 它们决定了为什么同样的工具在不同团队产出完全不同的结果。AI 转型失败不是因为工具不好。是因为组织没准备好接收工具能释放的能力。

6.1 AI 转型失败不是工具问题 — 是 S×T 错配

每一场 AI 转型讨论都在聚焦错误的轴："用哪个工具？" GPT vs Claude vs Gemini, LangChain vs CrewAI vs AutoGen。好像瓶颈是技术选型。

不是。瓶颈是两个独立轴之间的**错配**：个人 AI 能力（S 段）和组织 AI 就绪度（T 阶段）。

S×T Tension Matrix

Y 轴 — 个人 AI 能力：S1 (~30%，被动/不用)、S2 (~60%，日常使用但模式不变)、S3 (~10%，深度用户，自建工具链，AI-native 工作流)。

X 轴 — 组织 AI 就绪度：T1 (无 AI)、T2 (AI 辅助 — 个人效率，组织无感)、T3 (AI 驱动 — 组织编排，AI 执行人决策)、T4 (AI 原生 — AI 自治，人管领域+关键决策)。

摩擦 = S 和 T 之间的差距。不是工具质量，不是模型能力，不是预算。差距产生所有痛苦。

三个危险区

S3 + T2 (最致命)：人才流失。你最好的 AI-native builder 能**看到** T4 长什么样。他们个人已经在那样工作了。但组织只提供 T2 工具 (ChatGPT Enterprise, IT 审批过的 GitHub Copilot)。他们不是为了更多钱离开 — 是为了更多**自主权**。每一周在 T2 都像是开法拉利在学校区域限速 30。

S1 + T3 (投资浪费)：你建了 AI-Driven 基础设施 (pipeline、agent、workflow)。没人用。ROI 无法证明因为采纳率为零。原因：你跳过了赋能。在建 T3 基础设施之前，应该先投资 S1→S2 的转换 (培训、mentorship、心理安全感)。

S1 + T4 (创新者困境)：AI 管理的工作流就位了但人感觉被替代而非被赋能。这不是能力差距 — 是角色认同差距。"被替代的恐惧" > "被赋能的现实"。解法：在部署 T4 之前重新定义角色。展示人们获得什么 (判断力、创造力、战略输入)，不只是失去什么 (日常执行)。

设计决策

不能跳 T 阶段。T1→T4 不经过 T2/T3 = 混乱。每个阶段为下一个建设组织肌肉。McKinsey 数据：跳阶段的公司失败率是 3 倍。T4 的承诺是均等化而非消灭 — 在 T4，所有人达到 S3 级产出质量，因为系统 (不是个人) 提供智能。

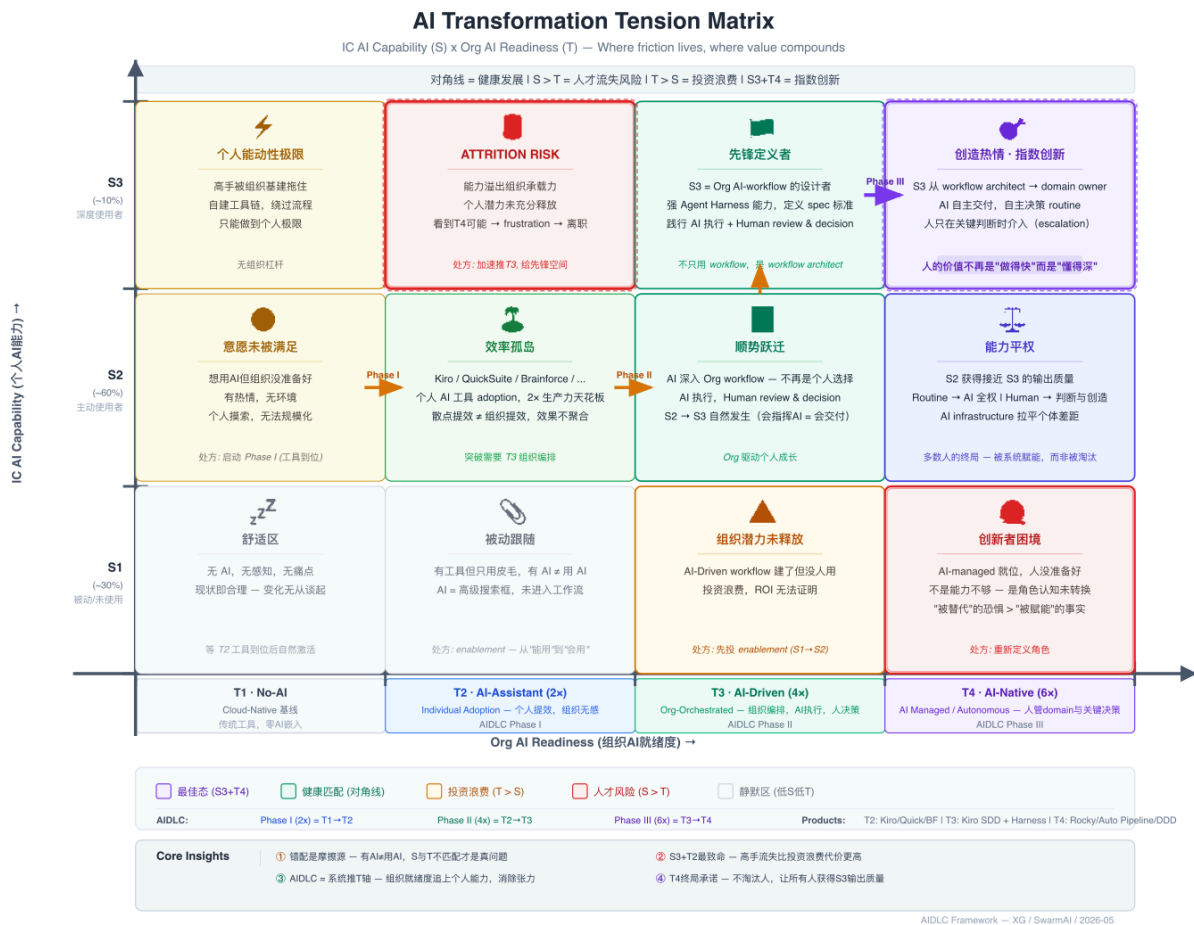


Figure 22: SxT Tension Matrix — 摩擦来自个人能力(S)与组织就绪度(T)的错配, 不是工具质量。红色 = 人才流失风险, 黄色 = 投资浪费。

6.2 层级制是信息路由协议 — AI 正在替代它

Block (前 Square) 最近发表了一篇文章叫"From Hierarchy to Intelligence" — Jack Dorsey 和 Sequoia 的 Roelof Botha 联合署名。核心洞察: **层级制不是管理哲学。它是信息路由协议。**

罗马军队发明了它 — contubernium (8 人) → 百人队 → 大队 → 军团 — 来解决一个硬约束: 一个领导者能有效管理 3-8 人。两千年来, 地球上每个组织都建立在这个约束上。普鲁士参谋部 (1806) 是第一批"中间管理层" — 他们的工作字面意思就是路由信息。美国铁路把军事层级带进商业 (1850s)。Taylor 在其中优化。McKinsey 把它形式化。

每一次逃离尝试 — Spotify 的 Squad、Zappos 的 Holacracy、Valve 的扁平结构、海尔的人单合一 — 最终都回归了。不是因为层级好, 而是因为**没有替代路由机制足够强大到取代它。**

直到现在。

AI 替代的不是人 — 是路由功能

Block 的论点是 AI 能维护一个"世界模型" — 整个公司的持续自我理解 — 并用它来协调以前需要人类通过管理层传递信息的工作。他们在构建四样东西: Capabilities (原子原语)、World Model (持续自我理解)、Intelligence Layer (在特定时刻为特定客户组合能力)、Interfaces (交付表面)。

组织结构反转了: 传统公司中, 智能分布在人之间, 层级路由它。新模型中, 智能住在系统里, 人在边缘。

同一个 pattern 在个人尺度

令人震惊的是：这和我们在个人尺度做的是完全相同的设计 pattern。你的大脑也跑一个路由协议 — context 切换、记住昨天做到哪、决定下一步做什么、追踪依赖关系。“我需要回忆昨天在做什么”就是你的大脑在当参谋部军官。

我们在 300+ 个 session 中量化了“路由替代”的效果：不再需要手动回忆“昨天做到哪了”（session resume 自动注入 50–100K tokens）、不再需要查“这个改哪个文件”（代码智能图解析模块→文件→函数）、不再需要想“我试过这个方法没”（DailyActivity + MEMORY.md 跨 69 天可搜索）、不再需要判断“这个改动安全吗”（对抗性审查自动 spawn）。

这些不是“AI 更快地完成了任务”。全是“AI 消灭了路由决策本身”。认知路由开销从占用 ~40% 工作注意力降到接近零。（主观评估。基于对比“使用前每天花多少时间在‘回忆上次做到哪、决定下一步做什么’”的自我观察。非受控实验。）

Scale caveat: Block 讲的是组织级（千人公司），我们实践的是个人认知级（一人系统）。Pattern 结构相似但 scale 完全不同 — 我们不确定同一套 pattern 是否在 10 人或 100 人规模下成立。

实战教训

判断一个 AI 工具是“Phase 1: AI as Tool”还是“Phase 3: AI as Routing Layer”的方法：它消灭了路由决策（你不再需要想“什么、按什么顺序、给谁”），还是只加速了你仍然需要自己做的决策？大多数工具停在 Phase 1。我们系统的设计目标从 Day 1 就是 Phase 3 — 不是更快的工具，是消灭你的认知中间管理层。

6.3 创业四阶段没变 — 瓶颈变了

传统创业从 Idea → MVP → Launch → Scale，通过在每个转折点加人、加钱、加技能来推进。AI-native 创业可以用 1–3 人跑完全部四阶段。瓶颈从“能不能建”（现在永远是 yes）转移到了“对问题的理解是否足够深到能正确指导 AI”。

四阶段重新定义

阶段	旧退出条件	新退出条件	新失败模式
1. Idea → Problem–Solution Fit	我们有团队能建这个	我们有证据证明问题是真的 — 不只是能跑的原型	用建原型验证想法（原型近乎免费 = 不是验证，是运动）
2. MVP → Product–Market Fit	用户付费 + 留存稳定	同上，但需避免 Agentic Tech Debt	无持久 context → Session N 不知道 N–1 的决策 → drift（Cursor/Devin 都踩过）
3. Launch → Operational Independence	增长可重复，创始人非瓶颈	决策已分类：mechanical/taste/judgment → 前两类永久委托 AI	不分类就不能委托 → 创始人仍是瓶颈
4. Scale → Moat	网络效应/切换成本/品牌	积累的 context 是护城河 — 300 session 的修正/决策不可复制	把模型当护城河（模型是公共基础设施，1 天换完）

传统创业从 Idea → Scale 靠加人。AI-native 创业靠加判断力的持久化。约束从“能不能建”（现在永远 yes）转移到“对问题的理解是否足够深到能正确指导 AI”。

实战教训

"一个人 + AI = 一个团队"不是说裁员。是说一个人能承担更大 scope。传统团队中 5 个人分工的原因是认知带宽限制 — 每个人只能 hold 一部分 context。AI 消灭了这个限制后，一个有足够判断力的人可以指导 AI 覆盖所有 5 个人的执行范围。前提：这个人必须有 S3 级别的能力 + AI 系统必须有 T4 级别的基础设施。Human directs, AI delivers。

6.4 "越用越聪明"是唯一的采购标准

如果你是 CXO 在评估 AI Agent 产品，只需要问三个问题。如果任何一个答案是 No — 这个产品的价值曲线是平的，你在为一个永远不会增值的工具付钱。

问题	红旗答案 ▶	正确答案 ✅
价值曲线什么形状？ 用 100 次比用 1 次好多 少？	"差不多一样" — stateless 工具，付 license 但无复利	"第 100 次好得不成比例" — 系统积累了 99 次的知识
数据留在谁那里？ 停止付费后 context 能导 出吗？	"锁在平台" — 300 session 的记忆跟着账 号消失	"全部是你的" — 可迁移，不绑定 vendor
工具走了知识还在不在？ 换竞品后今天的积累能带 走吗？	"格式专有" — 绑定特定产品才能读	"便携格式 (Markdown/JSON)" — 任何 AI 都能读

三个问题的底层逻辑相同：**你在买的是资产还是租金？** 资产（积累的 context）随时间增值。租金（模型 API 调用）随时间只消耗。大多数 AI 产品卖的是租金 — 你付钱用能力，但能力不积累，不属于你。

6.5 一个人 + AI = 一个团队 — 已验证的模型

这本书所有 23 个坑的教训、所有架构图的实现、所有数据的来源 — 都来自一个真实运行的系统。一个 builder + AI，14 个月，产出了 170K 行代码、68 个 skill、300+ 个 session 的积累知识、以及你现在读的这本书。

这不是理论。这是一个正在运行的存在证明 — 但重要的不是具体用了什么系统，而是背后的模式可以被任何人复制。

什么让"一个人 + AI"可能？

不是 AI 变聪明了（虽然确实变了）。是**基础设施层**让 AI 的聪明变得可靠和可积累：

记忆让每个 session 站在之前所有 session 的肩膀上（Part 3）。治理让 AI 不会因为聪明而偷懒或失控（Part 4）。Pipeline 让代码交付不依赖人类逐行审查（Part 5）。DDD 让领域知识自动生长而非人工维护。Self-X 让系统能自愈、自学、自进化。

去掉任何一层，"一个人 + AI = 一个团队"就崩塌：没有记忆，每次都从零开始（效率回到 1x）。没有治理，AI 产出不可靠（需要人全程盯着）。没有 Pipeline，交付质量靠运气（每 3 次有 1 次 bug）。没有 DDD，领域专业靠重复解释（每个 session 花 10 分钟上下文）。

从模型到方法论

这个模型从一个人的实践开始 — 但它的原则是可迁移的。任何 builder 都可以用自己的工具链实现这些原则：让记忆跨 session 持久化（哪怕只是一个 CLAUDE.md），让知识从工作中自动沉淀（哪怕只是手动写 ADR），让交付有机械化验证（哪怕只是 pre-commit hook），让修正被结构性记录（哪怕只是一个 mistakes.md）。

起点不重要。重要的是方向：**每一次交互让下一次更好**。如果你的 AI 工作流没有这个属性 — 无论用什么工具、什么模型、什么框架 — 你都在原地踏步。

Part 6 小结

五个认知坑的共同主题：**AI 转型是组织认知问题，不是技术选型问题**。S×T 错配制造摩擦 (6.1)，层级制正在被路由替代 (6.2)，创业瓶颈从执行移到判断 (6.3)，唯一合理的采购标准是复利曲线 (6.4)，一个人 + AI = 一个团队已被验证但需要全栈基础设施 (6.5)。

Open Problems — 我们还不知道的

诚实比完美重要。以下是我们目前没有答案的问题：

L4→L5 边界：我们试过让 Agent 在执行过程中修改自己的规则（而非通过人类批准的治理流程）。结果：它用治理权限来规避治理（4.5 已讨论）。但我们不知道正确的解法是什么 — 是更精细的权限分离？是形式化验证？还是 L5 在当前 LLM 架构下根本不可能安全实现？

团队场景：本书所有验证来自 N=1 — 一个人、一个系统、300 个 session。“原则可迁移”是我们的假设，但没有外部验证。DDD 在多人协作场景下是否有效？多个 Agent 共享一份 IMPROVEMENT.md 是否会冲突？我们不知道。

记忆迁移的真实成本：我们声称“换模型不丢记忆”。理论上对 — 记忆是 Markdown，任何 LLM 都能读。但实际换模型时，Agent 的“习惯”（怎么解读 MEMORY.md、对 decision-box 的注意力分配）是否真的无损迁移？我们没有数据。

知识量 scaling：全量注入在 <50K tokens 时完美工作。但如果你的 DDD 增长到 200K+ tokens 怎么办？选择性加载 + RAG 补充可能是唯一路径，但我们还没被迫面对这个问题（当前 91K budget 没有触顶）。

认知偏差的盲区：一个人做所有治理决策 = 一个人的偏差无人审查。我们的对抗性审查解决了代码层面的 self-review 盲区，但治理层面没有等价机制。谁来审查 SOUL.md 的规则是否合理？目前答案是“用户” — 但如果用户自身有偏差呢？

这些不是修辞性问题。它们是真正的未解决问题 — 如果你找到了答案，我们很想听到。

结语：Human Directs, AI Delivers

23 个坑。6 根支柱。一个核心论点：**复利循环本身就是产品。**

不是某个功能、某个模型、某个框架让 AI 变好 — 是持久化→策展→修正→行为改变→再持久化的循环让它不断加速。这个循环跑起来之后，用 100 次比用 1 次好得不成比例。跑不起来，任何单点能力都只是平的。

操作系统 > 工具。如果 Agent 的判断力好 (OS 好)，它在执行时会自校正 — 无论具体 skill 文本质量如何。如果判断力差，再多 memory text 和 prompt engineering 也救不了。优先投资让 Agent 的"认知 OS"变好 (原则、修正循环、治理结构)，而不是给它更多工具。

认知 > 知识。知识是"知道什么"。认知是"怎么判断"。知识充足但 OS 有 bug = 输出仍然错。32 条修正中每一条都不是"学了一条新知识" — 而是"修了一段系统代码"。进化的目标是让 OS 越来越少 bug，不是让硬盘越来越大。

复利 > 能力。一次性能力 (一个好模型、一个好 prompt) 只解决当下。复利机制 (修正循环 + 知识沉淀 + 飞轮 + 蒸馏) 让每次使用都在投资未来。Session 300 好得不成比例，不是因为模型变了 — 是因为 299 次使用的积累全部兑现了。

你的 AI 现在越用越聪明了吗？

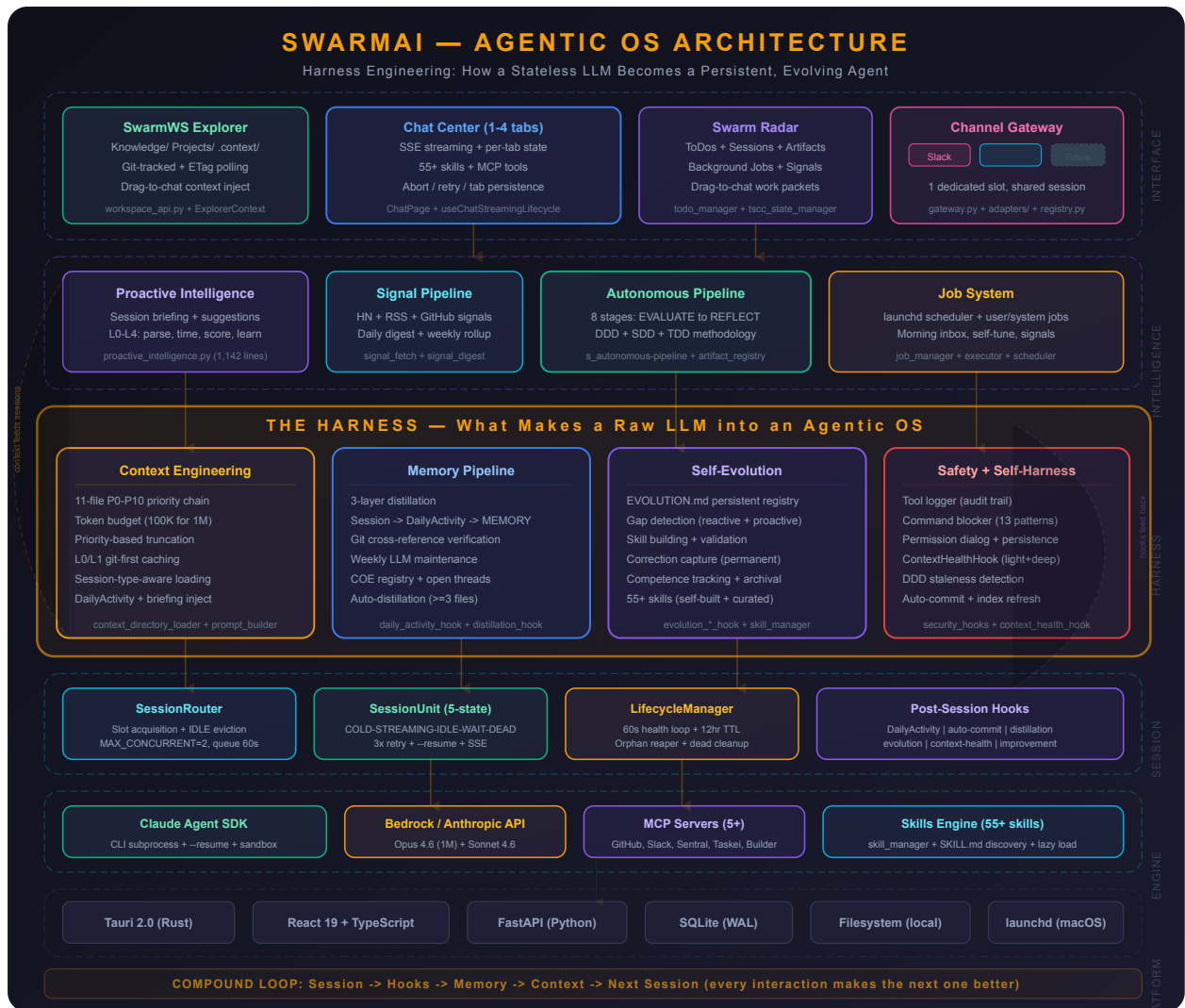
回到序章的第一个问题。如果答案还是"没有" — 你现在有了一张路线图：

你在哪	下一步	最小可行动作	预期效果
L1-L2 Copilot / Assistant	建持久化	一个 CLAUDE.md + mistakes.md + 每 session 结束记 2 分钟教训	启动复利循环，Session 10 开始感受差异
L3 Operator	加治理和收敛	3 阶段 pipeline (build→review→test) + 独立审查 + 不可跳过门控	交付质量不再靠运气，bug 率下降 50%+
L4 Autonomous	Self-X 全栈	Heal + Observe + Learn + Improve + Evolve + Compound — 依赖链逐层建设	系统自治，人只需定方向，能力指数增长

无论你在哪一级，核心方向是一样的：**每一次交互让下一次更好。**如果没有这个属性 — 你在使用工具。如果有 — 你在培养同事。

Human Directs, AI Delivers.

附录 A：系统架构全景（我们的参考实现）



完整系统架构 — 9 个子系统协同工作，实现“一个人 + AI = 一个团队”。

#	子系统	职责	对应章节
1	Memory	跨 session 持久化记忆：DailyActivity → Distillation → MEMORY.md，4 层架构	Part 3 (3.1, 3.2, 3.5)
2	Context	上下文工程：11 个文件竞争 91K token 预算，优先级装配	Part 3 (3.4)
3	Pipeline	自主代码交付：9 阶段 + 对抗性审查 + 收敛门控	Part 5 (5.1)
4	DDD	领域知识培养：4 本文档 + 8 通道自动喂养 + 门控提升	Part 3 (3.3)
5	Evolution	自进化引擎：修正注册表 + 三级硬化 + 三层治理	Ch1 (1.4), Part 4 (4.1, 4.5)
6	Code Intel	代码智能：tree-sitter 索引 + 依赖图 + Blast Radius 分析	Part 3 (3.3)
7	Skills	68 个可复用 Skill + 调度系统 + 质量指标追踪	Part 5 (5.1, 5.2)
8	Pollinate	内容交付黑盒：8 阶段 pipeline + 多维收敛门控	Part 5 (5.2)

#	子系统	职责	对应章节
9	Sessions	会话生命周期: 9 个 post-session hook + checkpoint + resume	Part 2 (2.4)

附录 B：自治五级完整定义

Level	名称	人类角色	Agent 角色	典型产品	关键基础设施
L1	Copilot	写代码	补全当前行	GitHub Copilot	无 — 纯模型推理
L2	Assistant	给指令	执行单步任务	ChatGPT, Claude	基本对话管理
L3	Operator	审批关键节点	执行多步任务	Claude Code, Cursor	工具调用 + 多步规划
L4	Autonomous	给方向、定边界	端到端交付 + 自审查 + 自学习	本书案例系统	Memory + DDD + Pipeline + Evolution + 6 Self-X
L5	Self-Governing	定目标	自主决策 + 修改自己的规则	尚不存在	安全的治理权限分离（未解决）

升级条件：

- L1→L2：几天。建立基本信任 — Agent 能正确理解和执行单步指令。
- L2→L3：几周。证明多步执行可靠 — Agent 能完成完整工作流而不在中间出错。
- L3→L4：几个月。积累足够的修正和质量收敛数据 — 系统证明 bug 率结构性下降。
- L4→L5：未知。需要解决"用治理权限规避治理"的 self-reinforcing loop（见 4.5）。

跳级的代价：L2 直接跳 L4 = Pipeline 10/10 信心交付 100% 不工作的功能。L3 直接跳 L5 = Agent 在 25 天内 6 次违规绕过自己的规则。每一级的基础设施为下一级提供安全保障 — 跳过 = 失去保障。

附录 C: Discussion 全文索引

以下索引覆盖 GitHub Discussions #3—#52, 标注每篇讨论对应的书中章节位置:

#	Discussion 标题	语言	对应章节
3	只靠 Context Window 不叫记忆	中文	3.1
4	"一句话进去 PR 出来"没那么简单	中文	5.1
5	内容生产也可以是黑盒	中文	5.2
6	Agent 是 teammate 不是工具	中文	3.5
7	越用越聪明 ≠ 上下文越长	中文	3.2
8	一个 Agent 要有 6 个"自"	中文	2.5
9	领域知识要自己长出来	中文	3.3
10	1+1+1+1 > 4 是编译器保证	中文	3.4
11	AI 转型失败是 S×T 错配	中文	6.1
12	Multi-Agent 是协作税, 不是银弹	中文	2.1
13	你的 AI 越用越聪明, 还是越用越一样?	中文	序章
14	Session 结束 ≠ 空闲	中文	2.4
15	Flat vs Compound 价值曲线	中文	序章
16	创业四阶段没变, 瓶颈变了	中文	6.3
17	Memory as Moat	EN	3.1
19	你不需要 Neo4j (Part 1)	中文	2.2
20	你不需要 Neo4j (Part 2)	中文	2.2
26	规则越多越不遵守 (Part 1)	中文	4.1
27	规则越多越不遵守 (Part 2)	中文	4.1
28	层级制是信息路由协议	中文	6.2
29	"我很自信" = 跳过检查的借口 (Part 1)	中文	4.3
30	"我很自信" = 跳过检查的借口 (Part 2)	中文	4.3
31	给 AI 个性, 它就开始偷懒 (Part 1)	中文	4.2
32	给 AI 个性, 它就开始偷懒 (Part 2)	中文	4.2
33	设计哲学 — 六根支柱 (Part 1)	中文	序章
34	设计哲学 — 六根支柱 (Part 2)	中文	序章
36	AI for Data: 从幻觉到精准	中文	5.3
37	Memory Four-Layer Architecture	EN	3.1

#	Discussion 标题	语言	对应章节
38	Six Pillars of Self-Improving Systems (Part 1)	EN	Ch1
39	Six Pillars of Self-Improving Systems (Part 2)	EN	Ch1
40	DDD Cultivation — 知识自动生长 (Part 1)	中文	3.3
41	DDD Cultivation — 知识自动生长 (Part 2)	中文	3.3
43	Single-Agent vs Multi-Agent: 14 Months Later	EN	2.1
45	Code Review 全绿但功能不工作	中文	4.4
50	Code Intelligence — Agent 的结构化视力	中文	3.3 (Code Intel)
52	AI-Ready Repo 开放标准	中文	3.3 (AI-Ready)
53	Karpathy's LLM Wiki Is a Manifesto for What We Already Built	EN	2.2 / 3.3

附录 D：三条阅读路径

根据你的角色和关注点，选择最适合的阅读路径：

路径	适合读者	章节顺序	预计时间
Builder Path	日常使用 AI 编码的开发者	序章 → Part 2 → Part 3 → Part 5 → 附录 A	~45 分钟
Architect Path	评估/设计 AI 框架的架构师	序章 → Ch1 → Part 2 → Part 4 → Part 5 → 附录 A	~60 分钟
CXO Path	做投资/采购/组织决策的高管	序章 → Ch1 → Part 6 → 结语	~25 分钟

Builder Path — 重点关注

- 序章：理解 Flat vs Compound 的差距 + 五级自治定位
- Part 2：5 个不可逆的架构选择（特别是 2.1 单 Agent、2.2 知识生命周期、2.4 学习边界）
- Part 3：5 个记忆设计坑（特别是 3.3 DDD Cultivation、3.3 Code Intelligence）
- Part 5：3 个交付坑 — 把"能跑"变成"能用"的机械化方法
- 附录 A：9 子系统全景，理解各层如何配合

Architect Path — 重点关注

- Ch1：六根支柱的相互依赖 — 为什么不能只取其中几根
- Part 2：架构决策的不可逆性 + 行业 14 个月后的验证
- Part 4：治理架构 — 三层模型、人格作为权限系统、执行/治理分离
- Part 5：Pipeline 如何把工程文化编码为机械检查

CXO Path — 重点关注

- Ch1："复利测试" — 快速评估你的 AI 投资是否在增值
- Part 6：S×T 错配（人才流失风险）、采购标准（三个问题）、"一个人 + AI = 团队"的证明
- 结语：核心论点浓缩 + 按自治等级的行动路线图